

# 數位控制系統第二章：離散時間系統與 z 轉換筆記

本文件彙整教材第 2 章「Discrete-Time Systems and the z-Transform」的完整內容，涵蓋離散時間系統的描述方式、z 轉換的定義與性質、差分方程式的求解、反 z 轉換的四種方法、模擬方塊圖與訊號流程圖、狀態變數模型的建立與轉換，以及狀態方程式的解。每個公式均附上推導過程、符號定義與對應的 MATLAB 實作，並在需要處以「前置知識」「補充說明」標示額外內容。

本章原文 PDF 為掃描辨識文字，部分數值在 OCR 過程中出現錯誤（例如小數點位數、上下標遺失）。本筆記在轉寫時已交叉驗證數學一致性並訂正，訂正處會以「 勘誤」標示。

## 一、離散時間系統的基本概念 (2.1–2.2)

### 1. 為什麼需要離散時間系統的描述方式？

連續時間系統（continuous-time system）的行為由微分方程式描述；離散時間系統（discrete-time system）的行為則由差分方程式（difference equation）描述。這個差異的根源在於：數位電腦只能在固定的取樣瞬間（ $kT$ ， $k = 0, 1, 2, \dots$ ）讀取與輸出數值，無法像類比電路一樣連續地微分或積分訊號。

對應地：

| 連續時間系統                    | 離散時間系統            |
|---------------------------|-------------------|
| 微分方程式                     | 差分方程式             |
| 拉普拉斯轉換（Laplace transform） | z 轉換（z-transform） |
| 轉移函數 $G(s)$               | 轉移函數 $G(z)$       |

### 2. 從類比 PI 控制器到數位差分方程式

考慮一個類比（連續時間）的比例積分（PI）控制器，其輸出為：

$$m(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau$$

其中  $e(t)$  為控制器輸入（誤差訊號）， $m(t)$  為控制器輸出， $K_P$ 、 $K_I$  為設計增益。若想用數位電腦（A/D → 電腦 → D/A）取代這個類比電路，電腦只能用**數值積分**逼近上式中的積分項。採用最簡單的**矩形法則**（rectangular rule）數值積分：把積分想像成用一系列窄矩形面積相加來逼近曲線下面積，則若  $x(kT)$  代表  $e(t)$  在  $t = kT$  時的數值積分近似值，可以寫成遞迴關係：

$$x(kT) = x[(k-1)T] + Te(kT)$$

其中  $T$  為取樣週期（數值演算法的步階大小，單位：秒）。這就是一個**一階差分方程式**——因為方程式中只用到「上一個取樣瞬間」的訊號。代入控制律後，數位 PI 控制器變成：

$$m(kT) = K_P e(kT) + K_I x(kT)$$

### 3. 一般形式的差分方程式

為方便書寫，以下把  $T$  省略（理解為  $x(k)$  即  $x(kT)$ ）。一階線性非時變差分方程式的一般形式為：

$$x(k) = b_1 e(k) + b_0 e(k-1) - a_0 x(k-1)$$

推廣到  $n$  階線性非時變差分方程式：

$$x(k) = b_n e(k) + b_{n-1} e(k-1) + \cdots + b_0 e(k-n) - a_{n-1} x(k-1) - \cdots - a_0 x(k-n)$$

對照連續時間  $n$  階系統的微分方程式：

$$y(t) = d_n \frac{d^n e(t)}{dt^n} + \cdots + d_1 \frac{de(t)}{dt} + d_0 e(t) - c_n \frac{d^n y(t)}{dt^n} - \cdots - c_1 \frac{dy(t)}{dt}$$

可以看出兩者的結構完全對應：微分運算 → 差分運算，連續時間的「歷史導數項」對應離散時間的「歷史取樣值」。

實現這個差分方程式的數位電腦程式（或專用硬體），本質上就是一個**數位濾波器**（digital filter）；系統設計者要決定三件事：(1) 取樣週期  $T$ ；(2) 差分方程式的階數  $n$ ；(3) 濾波器係數  $a_i, b_i$ ，使控制系統達到期望的特性。

**補充說明（工程背景）：**教材提到一個真實案例——美軍航艦自動降落系統的數位控制器，取樣頻率 25 Hz（即  $T = 0.04$  s），控制器為 11 階，為了避免電腦

運算捨入誤差造成系統誤差過大，最小需要 32 位元的字長 ( word length ) 。這說明差分方程式不只是數學形式，實作時還要考慮數值精度問題。

## 4. 宏觀補充：為什麼要用差分方程式取代微分方程式？

這一節把「差分方程式」放回更大的背景裡回答三個問題：它是什麼、為什麼要引入它、它怎麼跟真實的物理世界對話。

### ( 1 ) 差分方程式是什麼？

差分方程式就是微分方程式的離散 / 數位版本。連續世界裡，時間  $t$  連續流動，我們用微分方程式描述系統 ( 物體的加速度、電容的充電率、馬達的轉速變化... ) ；但電腦沒有「連續時間」的概念，電腦的世界像動畫影格一樣一格一格跳動 (  $k = 0, 1, 2, \dots$  ) 。差分方程式描述的正是「現在這個取樣點的狀態」與「過去幾個取樣點的狀態」之間的遞迴關係。

類比 ( 對有程式背景的讀者 ) ：差分方程式本質上就是陣列 / 變數的更新邏輯。

數學寫法： $x(k) = 0.5 x(k-1) + e(k)$

```
x = 0.5 * prev_x + current_e;
```

這正是本節例 2.1 ( 矩形法則數值積分 ) 與一般式  $x(k) = b_1 e(k) + b_0 e(k-1) - a_0 x(k-1)$  在程式碼層次的樣子——每一次迭代就是把「舊值」乘上係數再加上「新輸入」，更新成「新值」。

### ( 2 ) 為什麼要引入差分方程式？( 幹嘛這麼麻煩？ )

答案很現實：微控制器 ( MCU ) 、CPU 或 FPGA 根本不會算真正的微積分。真實物理系統 ( 發電機、冷氣壓縮機、無人機... ) 都是連續運作的；若用傳統類比電路 ( 電阻、電容、運算放大器 ) 做控制器，可以直接實現連續的微積分，但現代工程為了彈性、成本與可程式化，幾乎都改用數位電腦控制物理系統。引入差分方程式的目的只有一個：把微積分「降級」成電腦只靠加減乘除就能算出來的演算法。

| 領域        | 數學工具                   | 處理方式                         | 軟體實作        |
|-----------|------------------------|------------------------------|-------------|
| 類比 / 真實世界 | 微分方程式 ( Differential ) | 積分 $\int$ 、微分 $\frac{d}{dt}$ | 無法直接寫成程式碼   |
| 數位 / 電腦世界 | 差分方程式 ( Difference )   | 累加 $\sum$ 、相減 $\Delta$       | for 迴圈、變數更新 |

### (3) 它跟物理系統有何關聯？( 虛擬與現實的橋樑 )

把整個控制迴路想像成一個不斷循環的四步驟過程 ( 對照第一章 Fig. 1-1 的閉迴路方塊圖 ) :

1. **物理系統 ( 類比 )** : 受控體 ( 例如馬達 ) 持續轉動 , 這是連續的物理行為 , 服從微分方程式。
2. **感測器與 A/D 轉換** : 電腦每隔取樣週期  $T$  ( 例如每 1 毫秒 ) 「拍一張照片」, 把連續訊號變成離散數值  $e(k)$ ——這就是第三章零階保持器 ( ZOH ) 與取樣定理討論的起點。
3. **大腦計算 ( 差分方程式 )** : 電腦拿剛取樣到的  $e(k)$  , 配合記憶體裡存的上一步歷史值  $x(k-1)$  , 代入差分方程式做加減乘除 , 算出下一步輸出  $m(k)$ ——這正是本節模擬圖 ( 見「四、模擬方塊圖」) 用移位暫存器畫出來的過程。
4. **D/A 轉換與執行** : 電腦把  $m(k)$  轉回連續電壓訊號送給受控體 , 受控體 ( 物理系統 ) 因此改變行為 , 回到步驟 1 , 形成閉迴路。

**小結** : 真實的物理系統走的是微積分 , 但數位控制器只懂差分方程式。所以設計數位控制系統時 , 必須把物理世界的微分方程式透過\*\*取樣 ( sampling ) 與數值逼近 ( 例如本節的矩形法則 ) \*\*轉換成差分方程式 , 才能寫成程式碼或燒進晶片去控制真實世界——這也是為什麼下一節要引入 **z 轉換** : 它是差分方程式的「代數化」工具 , 地位完全對應連續世界裡拉普拉斯轉換之於微分方程式。( z 轉換中最基本的運算子  $z^{-1}$  , 物理意義就是「延遲一個取樣週期」, 直接對應差分方程式裡的  $x(k-1)$  , 也對應模擬圖裡的移位暫存器方塊——三者其實是同一件事的三種寫法 , 本章稍後會反覆用到這個對應關係。)

---

## 二、z 轉換的定義 (2.3)

---

### 1. 它是什麼？為什麼需要它？

正如拉普拉斯轉換把微分方程式轉成代數方程式 , 方便求解連續時間系統 , **z 轉換**把差分方程式轉成代數方程式 , 方便求解離散時間系統。z 轉換是對一個**數列**  $\{e(k)\}$  定義的轉換 , 而不是對連續函數定義的。

### 2. 定義

**單邊 z 轉換** ( single-sided / ordinary z-transform ) 定義為 :

$$E(z) = \mathcal{Z}[\{e(k)\}] = \sum_{k=0}^{\infty} e(k)z^{-k}$$

也就是說， $E(z)$  是變數  $z^{-1}$  的冪級數，級數的每一項係數就是數列  $e(k)$  的值。反  $z$  轉換則透過複數積分定義：

$$e(k) = \mathcal{Z}^{-1}[E(z)] = \frac{1}{2\pi j} \oint_{\Gamma} E(z)z^{k-1} dz$$

其中  $\Gamma$  是包住  $E(z)z^{k-1}$  所有有限極點的封閉路徑（實際操作時很少直接算這個積分，通常查表或用留數定理，見第七節）。

若序列  $e(k)$  是由連續時間訊號  $e(t)$  每隔  $T$  秒取樣而來，習慣上把  $e(kT)$  簡寫為  $e(k)$ （ $T$  省略但意義仍在）。

前置知識（雙邊  $z$  轉換）：另有雙邊  $z$  轉換（double-sided，也稱生成函數）  
 $G[\{f(k)\}] = \sum_{k=-\infty}^{\infty} f(k)z^{-k}$ ，但本書全程只使用單邊  $z$  轉換。

### 3. 範例與 MATLAB 對照

例 2.1：已知  $E(z) = 1 + 3z^{-1} - 2z^{-2} + z^{-4} + \dots$ ，直接比對係數即可得  $e(0) = 1, e(1) = 3, e(2) = -2, e(3) = 0, e(4) = 1, \dots$ ——這就是  $z$  轉換「係數即數列值」性質最直接的體現。

例 2.2：已知  $e(k) = 1$ （所有  $k$ ），求  $E(z)$ 。由定義：

$$E(z) = 1 + z^{-1} + z^{-2} + \dots$$

這是等比級數，利用恆等式  $\frac{1}{1-x} = 1 + x + x^2 + x^3 + \dots$ （ $|x| < 1$ ），取  $x = z^{-1}$ ，得到封閉形式：

$$E(z) = \frac{1}{1-z^{-1}} = \frac{z}{z-1}, \quad |z^{-1}| < 1$$

例 2.3：已知  $e(k) = a^{kT}$ ，同樣用等比級數公式：

$$E(z) = \frac{1}{1-a^T z^{-1}} = \frac{z}{z-a^T}, \quad |a^T z^{-1}| < 1$$

**MATLAB 對照**：Symbolic Math Toolbox 的 `ztrans()` 可以直接算出符號  $z$  轉換。

```
syms k
ek = 1^k;
Ez = ztrans(ek)      % Ez = z/(z - 1)    對應例 2.2

syms k a T
ek = exp(-a*k*T);    % 注意：這裡示範一般的 e(k)=exp(-a*kT) 形式
Ez = ztrans(ek)      % Ez = z/(z - exp(-a*T))    對應例 2.3
```

函數介紹：**syms** ——宣告接下來出現的變數（如 **k**、**a**、**T**）為「符號變數」而非數值，這樣 MATLAB 就能做符號（代數）運算而不是數值運算，是 Symbolic Math Toolbox 的起點。

函數介紹：**ztrans(expr)** ——對符號表達式 **expr**（以 **k** 為自變數的數列  $e(k)$ ）計算其單邊  $z$  轉換  $E(z)$ ，回傳值預設以 **z** 表示。這正是公式  $E(z) = \sum_{k=0}^{\infty} e(k)z^{-k}$  的符號運算實作。

數學  $\leftrightarrow$  MATLAB 對照：

- 數學上的求和  $\sum_{k=0}^{\infty} e(k)z^{-k} \leftrightarrow \text{ztrans}(ek)$  一行完成
- 數學上的封閉形式  $\frac{z}{z - a^T} \leftrightarrow \text{MATLAB 輸出 } z/(z - \exp(-a*T))$ （因為  $a^T = e^{T \ln a}$ ，符號引擎習慣以指數形式表示）

## 三、 $z$ 轉換的性質 (2.4)

這些性質是求解差分方程式與反  $z$  轉換的核心工具。

### 1. 線性（加法、減法、乘常數）

$$\mathcal{Z}[e_1(k) \pm e_2(k)] = E_1(z) \pm E_2(z), \quad \mathcal{Z}[\alpha e(k)] = \alpha E(z)$$

證明：直接把求和拆開即可，因為  $\sum_{k=0}^{\infty} [e_1(k) \pm e_2(k)]z^{-k} = \sum e_1(k)z^{-k} \pm \sum e_2(k)z^{-k}$ 。這兩條合稱**線性性質**。

### 2. 實數平移定理（Real Translation）

設  $n$  為正整數， $E(z) = \mathcal{Z}[e(k)]$ ，則：

$$\mathcal{Z}[e(k-n)u(k-n)] = z^{-n}E(z) \quad (\text{延遲/右移})$$

$$\mathcal{Z}[e(k+n)u(k)] = z^n \left[ E(z) - \sum_{k=0}^{n-1} e(k)z^{-k} \right] \quad (\text{超前/左移})$$

其中  $u(k)$  是離散單位步階函數： $u(k) = 1 (k \geq 0)$ ， $u(k) = 0 (k < 0)$ 。

**物理意義：**右移（延遲）不會遺失任何數列資訊，所以  $z^{-n}E(z)$  形式簡單；但左移（超前）會把數列最前面的  $n$  個值「甩出視窗外」，因此公式中要扣掉

$\sum_{k=0}^{n-1} e(k)z^{-k}$  這些即將被丟失的項來做補償。下表具體示範範位移 2 個取樣週期的效果：

| $k$ | $e(k)$ | $e(k-2)$ | $e(k+2)$ |
|-----|--------|----------|----------|
| 0   | 2.0    | 0        | 1.3      |
| 1   | 1.6    | 0        | 1.1      |
| 2   | 1.3    | 2.0      | 1.0      |
| 3   | 1.1    | 1.6      | ...      |
| 4   | 1.0    | 1.3      | ...      |

**例 2.5：**已知  $\mathcal{Z}[a^{kT}] = \frac{z}{z - a^T}$ ，則：

$$\mathcal{Z}[a^{(k-3)T}u[(k-3)T]] = z^{-3} \cdot \frac{z}{z - a^T} = \frac{1}{z^2(z - a^T)}$$

### 3. 單位脈衝函數與其轉換

定義離散單位脈衝函數（unit impulse / unit sample function）：

$$\delta(k-n) = \begin{cases} 1, & k = n \\ 0, & k \neq n \end{cases}$$

其  $z$  轉換非常簡單（把定義代入求和公式，只剩一項）：

$$\mathcal{Z}[\delta(k-n)] = z^{-n} \quad (n \geq 0)$$

### 4. 複數平移定理（Complex Translation）

$$\mathcal{Z}[a^{kT}e(kT)] = E(z) \Big|_{z \rightarrow z/a^T} = E(z/a^T)$$

**物理意義：**在時域乘上一個指數衰減（或成長）項  $a^{kT}$ ，對應到  $z$  域中把變數  $z$  換成  $z/a^T$ （座標縮放）。

**例 2.6：**已知  $\mathcal{Z}[kT] = \frac{Tz}{(z-1)^2}$ ，求  $\mathcal{Z}[kT a^{kT}]$ 。直接套用複數平移定理，把  $z$  換成  $z/a^T$ ：

$$\mathcal{Z}[kT a^{kT}] = \frac{T(z/a^T)}{(z/a^T - 1)^2} = \frac{T a^T z}{(z - a^T)^2}$$

```
syms a k T
ekT = (k*T)*exp(a*k*T);
Ez = ztrans(ekT)
% Ez = (T*z*exp(T*a))/(z - exp(T*a))^2
```

## 5. 初值定理 ( Initial Value Theorem )

$$e(0) = \lim_{z \rightarrow \infty} E(z)$$

**證明：**因為  $E(z) = e(0) + e(1)z^{-1} + e(2)z^{-2} + \dots$ ，當  $z \rightarrow \infty$  時所有含  $z^{-1}$  的項都趨近於 0，只留下  $e(0)$ ，由檢視即可得證。

## 6. 終值定理 ( Final Value Theorem )

$$\lim_{n \rightarrow \infty} e(n) = \lim_{z \rightarrow 1} (z-1)E(z)$$

**成立條件：**等號右側極限存在，且（將在第 7 章證明） $E(z)$  的所有極點都必須落在單位圓內，最多允許在  $z = 1$  處有一個單重極點。

**推導概念：**利用  $\mathcal{Z}[e(k+1) - e(k)]$  這個「一階差分」的  $z$  轉換，一方面用實數平移定理展開得到  $(z-1)E(z) - ze(0)$ ，另一方面直接展開級數並取  $z \rightarrow 1$  的極限會得到  $\lim_{n \rightarrow \infty} [e(n+1) - e(0)]$ ；令兩者相等，兩邊消去  $e(0)$  即得終值定理。

**例 2.7：**驗證初值/終值定理。已知  $e(k) = 1$ （例 2.2）， $E(z) = \frac{z}{z-1}$ 。

初值：

$$e(0) = \lim_{z \rightarrow \infty} \frac{z}{z-1} = \lim_{z \rightarrow \infty} \frac{1}{1-1/z} = 1 \checkmark$$

終值：

$$\lim_{k \rightarrow \infty} e(k) = \lim_{z \rightarrow 1} (z - 1) \cdot \frac{z}{z - 1} = \lim_{z \rightarrow 1} z = 1 \checkmark$$

## 7. z 轉換性質總表 ( Table 2-2 )

| 數列                           | 轉換                                                      |
|------------------------------|---------------------------------------------------------|
| $e(k)$                       | $E(z) = \sum_{k=0}^{\infty} e(k)z^{-k}$                 |
| $a_1 e_1(k) + a_2 e_2(k)$    | $a_1 E_1(z) + a_2 E_2(z)$                               |
| $e(k - n)u(k - n), n \geq 0$ | $z^{-n} E(z)$                                           |
| $e(k + n)u(k), n \geq 1$     | $z^n \left[ E(z) - \sum_{k=0}^{n-1} e(k)z^{-k} \right]$ |
| $a^{kT} e(k)$                | $E(z/a^T)$                                              |
| $k e(k)$                     | $-z \frac{dE(z)}{dz}$                                   |
| $e_1(k) * e_2(k)$ ( 摺積 )     | $E_1(z)E_2(z)$                                          |
| $e_1(k) = \sum_{n=0}^k e(n)$ | $\frac{z}{z - 1} E(z)$ ( 若 $e(\infty)$ 存在 )             |
| 初值                           | $e(0) = \lim_{z \rightarrow \infty} E(z)$               |
| 終值                           | $e(\infty) = \lim_{z \rightarrow 1} (z - 1)E(z)$        |

## 四、常用 z 轉換表與由 s 域求 z 轉換 (2.5)

### 1. 常用數列的 z 轉換 ( Table 2-3 · 一般取樣週期 $T$ )

取樣週期  $T$  不應該被隱藏假設成 1——只要  $e(t)$  是實際的連續時間訊號，取樣後的數列就是  $e(kT)$  ( 時間軸上真正的間隔是  $T$  秒 )，對應的 z 轉換也一定會顯式地含有  $T$  ( 尤其是指數與三角函數項， $T$  會出現在指數  $e^{-\alpha T}$  或角度  $\omega T$  裡 )。下表把每一列訊號的三種面貌都列出來：拉普拉斯轉換  $E(s)$  ( 連續時間系統分析用 )、取樣後的數列  $e(kT)$ ，以及對應的 z 轉換  $E(z)$ ——這也直接對應本節第 2 小節「由 s 域函數求 z 轉換」的查表法：只要拿到  $E(s)$  的部分分式，就能逐項對照這張表換成  $E(z)$ ，不需要再重新推導一次：

| 拉普拉斯轉換<br>$E(s)$                               | 連續時間訊號<br>$e(t) \cdot t \geq 0$ | 取樣後數列<br>$e(kT)$                | $z$ 轉換 $E(z)$                                                                                       |
|------------------------------------------------|---------------------------------|---------------------------------|-----------------------------------------------------------------------------------------------------|
| —                                              | $\delta(t)$                     | $\delta(k)$                     | 1                                                                                                   |
| —                                              | ( 延遲 $n$ 個取樣 )                  | $\delta(k - n)$                 | $z^{-n}$                                                                                            |
| $\frac{1}{s}$                                  | 單位步階 $u_s(t)$                   | 1                               | $\frac{z}{z - 1}$                                                                                   |
| $\frac{1}{s^2}$                                | $t$ ( 斜坡 )                      | $kT$                            | $\frac{Tz}{(z - 1)^2}$                                                                              |
| $\frac{2}{s^3}$                                | $t^2$                           | $(kT)^2$                        | $\frac{T^2 z(z + 1)}{(z - 1)^3}$                                                                    |
| $\frac{1}{s + \alpha}$                         | $e^{-\alpha t}$                 | $e^{-\alpha kT}$                | $\frac{z}{z - e^{-\alpha T}}$                                                                       |
| $\frac{1}{(s + \alpha)^2}$                     | $t e^{-\alpha t}$               | $kT e^{-\alpha kT}$             | $\frac{T e^{-\alpha T} z}{(z - e^{-\alpha T})^2}$                                                   |
| $\frac{\omega}{s^2 + \omega^2}$                | $\sin \omega t$                 | $\sin \omega kT$                | $\frac{z \sin \omega T}{z^2 - 2z \cos \omega T + 1}$                                                |
| $\frac{s}{s^2 + \omega^2}$                     | $\cos \omega t$                 | $\cos \omega kT$                | $\frac{z(z - \cos \omega T)}{z^2 - 2z \cos \omega T + 1}$                                           |
| $\frac{\omega}{(s + \alpha)^2 + \omega^2}$     | $e^{-\alpha t} \sin \omega t$   | $e^{-\alpha kT} \sin \omega kT$ | $\frac{z e^{-\alpha T} \sin \omega T}{z^2 - 2z e^{-\alpha T} \cos \omega T + e^{-2\alpha T}}$       |
| $\frac{s + \alpha}{(s + \alpha)^2 + \omega^2}$ | $e^{-\alpha t} \cos \omega t$   | $e^{-\alpha kT} \cos \omega kT$ | $\frac{z^2 - z e^{-\alpha T} \cos \omega T}{z^2 - 2z e^{-\alpha T} \cos \omega T + e^{-2\alpha T}}$ |

**符號說明：**「—」代表這一系列沒有對應的連續時間  $E(s)$ —— $\delta(k - n)$  是在  $z$  域直接定義的純離散延遲運算（見第二節「平移定理」），並非某個連續訊號取樣後才出現的結果，因此沒有  $s$  域對應項。

**與後文簡寫記號的關係：**本章其餘例題（例 2.8、第五、六節的反  $z$  轉換）常直接寫  $a^k \rightarrow \frac{z}{z - a}$ 、 $ka^k \rightarrow \frac{az}{(z - a)^2}$ 、 $\sin ak \rightarrow \frac{z \sin a}{z^2 - 2z \cos a + 1}$  這類「無  $T$ 」的簡化式。它們不是另一組公式，而是把上表中的  $e^{-\alpha T}$ （或  $\omega T$ ）整體重新命名為一個符號  $a$ （或角度  $a$ ）——數學上完全等價，只是省略了  $T$  以方便書寫，在單純處理離散序列（不追問其連續時間來源）時很常見。但只要是由  $s$  域

訊號取樣而來 ( 如下面「由  $s$  域函數求  $z$  轉換」與例 2.9 ) , 就必須把  $T$  明確寫出來, 不能省略, 因為  $T$  會直接決定極點位置  $e^{-\alpha T}$  在  $z$  平面上的實際數值。

**例 2.8 :** 用 MATLAB 驗證表中最後一列  $e^{-\alpha kT} \cos \omega kT$  的簡化記號  $a^k \cos bk$  ( 即  $a = e^{-\alpha T}$ 、 $b = \omega T$ 、暫不寫出  $T$  ) 的轉換 :

```
syms a b k
Ez = ztrans(a^k*cos(b*k));
pretty(Ez)
```

**函數介紹 :** `pretty(expr)` —— 把符號表達式以較接近手寫數學的排版方式印出 ( 分數用橫線、根號等 ) , 方便閱讀複雜的符號運算結果, 純粹是顯示用途, 不影響運算結果本身。

若把取樣週期  $T$  也考慮進去 ( 把  $k$  換成  $kT$  ) :

```
syms a b k T
Ez = ztrans(a^(k*T)*cos(b*k*T));
pretty(Ez)
```

得到 :

$$\mathcal{Z}[a^{kT} \cos bkT] = \frac{z^2 - a^T z \cos bT}{z^2 - 2a^T z \cos bT + a^{2T}}$$

## 2. 由拉普拉斯域 ( $s$ 域 ) 函數求 $z$ 轉換

在數位控制系統設計中, 系統函數常常先以  $s$  域 ( 連續時間 ) 給出, 因此需要一套方法把  $E(s)$  轉成  $E(z)$ 。標準做法是 :

1. 把  $E(s)$  做部分分式展開, 拆成一次極點的和。
2. 對每一項套用「 $\frac{1}{s+a}$  取樣後對應  $\frac{z}{z-e^{-aT}}$ 」這類 Laplace $\leftrightarrow$  $z$  對照表 ( 見附錄 VI ) , 逐項轉成  $z$  域並相加 ( 利用線性性質 ) 。

**例 2.9 :** 求  $E(s) = \frac{s^2 + 4s + 3}{s^3 + 6s^2 + 8s}$  的  $z$  轉換。

**第一步 :** 因式分解分母並做部分分式展開

$$E(s) = \frac{s^2 + 4s + 3}{s(s+2)(s+4)} = \frac{K_0}{s} + \frac{K_1}{s+2} + \frac{K_2}{s+4}$$

用留數法逐一求係數 ( $K_i = (s - p_i)E(s)|_{s=p_i}$ ) :

$$K_0 = \frac{0^2 + 4(0) + 3}{(0 + 2)(0 + 4)} = \frac{3}{8} = 0.375$$

$$K_1 = \frac{(-2)^2 + 4(-2) + 3}{(-2)(-2 + 4)} = \frac{-1}{-4} = 0.25$$

$$K_2 = \frac{(-4)^2 + 4(-4) + 3}{(-4)(-4 + 2)} = \frac{3}{8} = 0.375$$

⚠ 勘誤：原書掃描文字在  $K_1$  處顯示為「0.025」，經上式重新驗算應為 0.25 (極可能是 OCR 遺漏小數位)，本筆記已訂正。

所以：

$$E(s) = \frac{0.375}{s} + \frac{0.25}{s + 2} + \frac{0.375}{s + 4}$$

第二步：逐項查表轉成  $z$  域。 $\frac{1}{s + a}$  (取樣後) 對應  $\frac{z}{z - e^{-aT}}$ ，因此：

$$E(z) = 0.375 \frac{z}{z - 1} + 0.25 \frac{z}{z - e^{-2T}} + 0.375 \frac{z}{z - e^{-4T}}$$

取  $T = 0.1 \text{ s}$ ，則  $e^{-2(0.1)} = e^{-0.2} = 0.8187$ ， $e^{-4(0.1)} = e^{-0.4} = 0.6703$ ：

$$E(z) = \frac{0.375z}{z - 1} + \frac{0.25z}{z - 0.8187} + \frac{0.375z}{z - 0.6703}$$

通分整理成單一有理式：

$$E(z) = \frac{z^3 - 1.658z^2 + 0.6804z}{z^3 - 2.489z^2 + 2.038z - 0.5488}$$

MATLAB 作法一：`residue()` 手動走一遍部分分式流程

```

T = 0.1;
num = [1 4 3];
denom = [1 6 8 0]; % s^3+6s^2+8s，且不能有重根
n = length(denom);
Es = tf(num, denom) % 建立 s 域轉移函數，方便檢查

[r, p, k] = residue(num, denom); % 部分分式展開：r=留數，p=極點，k=直接項
for i = 1:n-1
    pz(i) = exp(p(i)*T); % 每個 s 域極點 p 對應 z 域極點 exp(p*T)
end

[numzz, denomz] = residue(r, pz, k); % 把「z 域的留數/極點」合併回多項式
numz = conv(numzz, [1 0]); % 乘上一個 z（因為查表結果都帶一個 z）
Ez = tf(numz, denomz, T)

```

**函數介紹：** `residue(num, den)` ——對有理多項式  $\frac{\text{num}(s)}{\text{den}(s)}$  做部分分式展開，回傳三個輸出：`r`（留數，對應每個極點的分子係數）、`p`（極點）、`k`（若分子階數  $\geq$  分母階數時的直接項，多項式除法的商）。這正是手算部分分式時「求  $K_0, K_1, K_2$ 」步驟的自動化版本。反過來呼叫 `residue(r, p, k)`（用留數與極點回推多項式）則可以把已知的「留數+極點」形式合併回一個有理多項式——程式中第二次呼叫就是利用這個特性，把每一項  $\frac{r_i}{z - \exp(p_i T)}$  加總、通分成一個多項式除以多項式的形式。

**函數介紹：** `conv(a, b)` ——計算兩個多項式係數向量的摺積（convolution），數學上摺積等於多項式相乘。這裡 `conv(numzz, [1 0])` 就是把 `numzz` 這個多項式乘上  $z$ （因為 `[1 0]` 代表多項式  $1 \cdot z + 0 = z$ ）。

**函數介紹：** `tf(num, den, T)` ——與連續時間 `tf(num, den)` 用法相同，只是多了第三個參數 `T`：當提供取樣週期 `T` 時，MATLAB 會建立一個離散時間轉移函數模型（自變數為  $z$  而非  $s$ ）。

把這個例子的數字實際帶入，逐一看每個變數長什麼樣：

`Es = tf(num, denom)` 顯示連續時間轉移函數：

```

Es =
      s^2 + 4 s + 3
      -----
      s^3 + 6 s^2 + 8 s

Continuous-time transfer function.

```

`[r, p, k] = residue(num, denom)` ——這就是手算「求  $K_0, K_1, K_2$ 」的自動化版本：

```
r =  
    0.3750  
    0.2500  
    0.3750  
  
p =  
     0  
   -2.0000  
   -4.0000  
  
k =  
    []      (空的：分子階數 2 < 分母階數 3，沒有多項式直接項)
```

對照手算結果：  $r(1)=0.375$  配  $p(1)=0$ ，正是  $K_0 = 0.375$  (極點在  $s = 0$ )；  
 $r(2)=0.25$  配  $p(2)=-2$ ，是  $K_1 = 0.25$  (極點在  $s = -2$ )；  
 $r(3)=0.375$  配  $p(3)=-4$ ，是  $K_2 = 0.375$  (極點在  $s = -4$ ) ——三個數字與本節第一步手算完全一致。(不同 MATLAB 版本回傳的列順序可能不同，但每一組「留數配極點」的數值配對是固定的，順序調換不影響最終結果。)

`for i=1:n-1; pz(i)=exp(p(i)*T); end` ( $T = 0.1$ ) 把每個  $s$  域極點映射成對應的  $z$  域極點：

```
pz =  
    1.0000    0.8187    0.6703
```

(依序對應  $e^{0 \times 0.1} = 1$ 、 $e^{-2 \times 0.1} = e^{-0.2} = 0.8187$ 、 $e^{-4 \times 0.1} = e^{-0.4} = 0.6703$  ——這正是本節「查表」用到的  $e^{-aT}$ 。)

`[numzz, denomz] = residue(r, pz, k)` ——用同一組留數 **r**、但換成  $z$  域極點 **pz**，反推合併回一個多項式除以多項式：

```
numzz =  
    1.0000   -1.6580    0.6804  
  
denomz =  
    1.0000   -2.4891    2.0379   -0.5488
```

`numz = conv(numzz, [1 0])` (乘上一個  $z$ ，因為查表結果  $\frac{z}{z-e^{-aT}}$  分子都帶一個  $z$ )：

```
numz =  
      1.0000   -1.6580    0.6804         0
```

`Ez = tf(numz, denomz, T)` 得到最終的離散轉移函數：

```
Ez =  
      z^3 - 1.658 z^2 + 0.6804 z  
      -----  
      z^3 - 2.489 z^2 + 2.038 z - 0.5488  
  
Sample time: 0.1 seconds  
Discrete-time transfer function.
```

跟本節第二步手算通分後的  $E(z) = \frac{z^3 - 1.658z^2 + 0.6804z}{z^3 - 2.489z^2 + 2.038z - 0.5488}$  一模一樣——手算部分分式與 MATLAB `residue()` 這條路徑，殊途同歸。

**MATLAB 作法二：先反拉普拉斯回到時域，再對  $t = kT$  取  $z$  轉換**

```
T = 0.1;  
syms s t k z  
Es = (s^2+4*s+3)/(s^3+6*s^2+8*s);  
et = ilaplace(Es); % 反拉普拉斯轉換：s 域 -> 時域 e(t)  
ekT = subs(et, t, k*T); % 代入 t = kT，得到取樣後的數列 e(kT)  
Ez = ztrans(ekT); % 對數列取 z 轉換  
pretty(Ez)
```

**函數介紹：**`ilaplace(expr)`——對符號表達式做反拉普拉斯轉換，把  $s$  域函數轉回時域函數  $e(t)$ ，是 `laplace()` 的反運算。

**函數介紹：**`subs(expr, old, new)`——把符號表達式 `expr` 中的變數 `old` 代換成 `new`。這裡把時間變數 `t` 換成 `k*T`，等於是「每隔  $T$  秒取一個樣本」的數學操作。

**數學 ↔ MATLAB 對照：**兩種 MATLAB 作法分別對應本節「先部分分式再查表」與「先反拉普拉斯到時域，再取樣、再做  $z$  轉換」兩條數學路徑，殊途同歸，得到相同的  $E(z)$ 。

## 五、差分方程式的求解 (2.6)

求解線性非時變差分方程式有三種基本技巧：

### 1. 古典法 ( Classical Approach )

類似求解線性微分方程式的「齊次解 + 特解」方法，本書不深入討論（留待狀態變數方法時介紹類似技巧）。

### 2. 逐次代入法 ( Sequential / Recursive Method )

這是數位電腦求解差分方程式最直接的方式：從  $k = 0$  開始，一步步往後代入。

例 2.10：求解  $m(k) = e(k) - e(k-1) - m(k-1)$ ,  $k \geq 0$ ，其中  $e(k) = 1$  ( $k$  為偶數)、 $e(k) = 0$  ( $k$  為奇數)，且  $e(-1) = m(-1) = 0$ 。

逐步代入：

$$m(0) = e(0) - e(-1) - m(-1) = 1 - 0 - 0 = 1$$

$$m(1) = e(1) - e(0) - m(0) = 0 - 1 - 1 = -2$$

$$m(2) = e(2) - e(1) - m(1) = 1 - 0 - (-2) = 3$$

$$m(3) = e(3) - e(2) - m(2) = 0 - 1 - 3 = -4$$

$$m(4) = e(4) - e(3) - m(3) = 1 - 0 - (-4) = 5$$

MATLAB 對應（可解到任意  $k$ ，這正是數位電腦擅長的重複運算）：

```
mkminus1 = 0; % m(k-1) 的初始值 m(-1)
ekminus1 = 0; % e(k-1) 的初始值 e(-1)
ek = 1;      % e(0)
for k = 0:20
    mk = ek - ekminus1 - mkminus1;
    [k, mk] % 顯示目前的 k 與 m(k)
    mkminus1 = mk; % 為下一輪迭代更新「上一步」的值
    ekminus1 = ek;
    ek = 1 - ek; % e(k) 在 0 與 1 之間交替
end
```

函數介紹：`for k = a:b ... end`——迴圈語法，讓  $k$  依序取  $a, a+1, \dots, b$ ，每次迭代執行迴圈內的敘述。這裡完美對應差分方程式「逐次代入」的數學過程：每一次迭代就是計算下一個時間點的  $m(k)$ 。

數學  $\leftrightarrow$  MATLAB 對照：

- 變數 `mkminus1`  $\leftrightarrow$  數學中的  $m(k-1)$
- 變數 `mk`  $\leftrightarrow$  數學中的  $m(k)$
- 迴圈本體的三行賦值  $\leftrightarrow$  把「這一輪算出的新值」變成「下一輪的舊值」，也就是時間往前推進一步

### 3. z 轉換法

考慮一般的  $n$  階差分方程式 ( 已知輸入數列  $\{e(k)\}$  ) :

$$m(k) + a_{n-1}m(k-1) + \cdots + a_0m(k-n) = b_ne(k) + b_{n-1}e(k-1) + \cdots + b_0e(k-n)$$

利用實數平移定理對兩邊取  $z$  轉換 ( 初始條件為零 ) :

$$M(z)[1 + a_{n-1}z^{-1} + \cdots + a_0z^{-n}] = E(z)[b_n + b_{n-1}z^{-1} + \cdots + b_0z^{-n}]$$

$z$  轉換把差分方程式變成代數方程式，解出  $M(z)$  :

$$M(z) = \frac{b_n + b_{n-1}z^{-1} + \cdots + b_0z^{-n}}{1 + a_{n-1}z^{-1} + \cdots + a_0z^{-n}} E(z)$$

再對  $M(z)$  取反  $z$  轉換即可得  $m(k)$  ( 反  $z$  轉換方法見第七節 ) 。

**例 2.11** : 用  $z$  轉換法重解例 2.10。差分方程式  $m(k) = e(k) - e(k-1) - m(k-1)$  取  $z$  轉換 :

$$M(z) = E(z) - z^{-1}E(z) - z^{-1}M(z) \implies M(z) = \left( \frac{z-1}{z+1} \right) E(z)$$

而  $e(k)$  是週期為 2 的方波 ( 1, 0, 1, 0, ... )，其  $z$  轉換為 :

$$E(z) = 1 + z^{-2} + z^{-4} + \cdots = \frac{1}{1 - z^{-2}} = \frac{z^2}{z^2 - 1} = \frac{z^2}{(z-1)(z+1)}$$

代入得 :

$$M(z) = \frac{z-1}{z+1} \cdot \frac{z^2}{(z-1)(z+1)} = \frac{z^2}{(z+1)^2}$$

用長除法把  $M(z)$  展開成  $z^{-1}$  的冪級數 ( 長除法細節見第七節「冪級數法」 ) :

$$M(z) = 1 - 2z^{-1} + 3z^{-2} - 4z^{-3} + \cdots$$

對照係數得  $m(0) = 1, m(1) = -2, m(2) = 3, m(3) = -4, \dots$ ，與例 2.10 逐次代入法得到的結果完全一致！這證明了  $z$  轉換法與逐次代入法是同一個問題的兩種解

法。

#### 4. 含非零初始條件的情況

以上推導都假設初始條件為零，得到的只是「強制響應」( forced response )。若要納入非零初始條件，把差分方程式中的  $k$  換成  $k + n$ ：

$$m(k + n) + a_{n-1}m(k + n - 1) + \cdots + a_0m(k) = b_ne(k + n) + \cdots + b_0e(k)$$

再利用實數平移定理 ( 超前形式 )：

$$\mathcal{Z}[m(k + i)] = z^i \left[ M(z) - m(0) - m(1)z^{-1} - \cdots - m(i - 1)z^{-(i-1)} \right]$$

這樣轉換出來的代數方程式就會自動包含初始條件  $m(0), m(1), \dots, m(n - 1)$ 。值得注意的是：對於  $n$  階差分方程式，「初始條件」指的是  $m(0), \dots, m(n - 1)$  這  $n$  個取樣值，而不是像微分方程式那樣的各階導數初始值——這是差分方程式與微分方程式在「初始條件」意義上的關鍵差異。

---

## 六、反 $z$ 轉換的四種方法 (2.7)

---

### 方法一：冪級數法 ( 長除法 )

若  $E(z) = \frac{N(z)}{D(z)}$  是兩個多項式的比值，直接用長除法把分母除進分子，展開成  $z^{-1}$  的冪級數：

$$E(z) = e_0 + e_1z^{-1} + e_2z^{-2} + \cdots$$

依定義，級數係數就是  $e(k)$  的值。

**例 2.12**：求  $E(z) = \frac{z}{z^2 - 3z + 2}$  的反  $z$  轉換。用長除法 ( 分子分母同除以最高次項整理成  $z^{-1}$  冪級數形式後相除 ) 得到：

$$E(z) = z^{-1} + 3z^{-2} + 7z^{-3} + 15z^{-4} + \cdots$$

對照係數： $e(0) = 0, e(1) = 1, e(2) = 3, e(3) = 7, e(4) = 15, \dots$ ，可以看出規律  $e(k) = 2^{k-1}$  ( $k \geq 1$ )。

**優點與限制**：這個方法簡單直接，總能算出任意有限個  $e(k)$  的數值；但通常無法直接得到  $e(k)$  關於  $k$  的封閉公式 ( 本例能看出  $2^{k-1}$  的規律純屬巧合、好認 )。若要得到

封閉公式，通常需要下面的部分分式法。

## 方法二：部分分式展開法

**核心技巧：**查表 ( Table 2-3、附錄 VI ) 時，注意到表中每個轉換式的分子都帶有一個  $z$ ，例如  $\frac{z}{z-a} \rightarrow a^k$ 。因此做部分分式展開前，應該先把  $\frac{E(z)}{z}$  ( 而不是  $E(z)$  本身 ) 展開，這樣每一項才會自然帶有  $\frac{1}{z-a}$  的形式，最後再乘回一個  $z$ ：

$$\mathcal{Z}^{-1} \left[ \frac{z}{z-a} \right] = a^k$$

這是全書最常見的一個轉換對，因為指數數列  $\{a^k\}$  是離散系統最基本的行為模式。

**例 2.13：**續例 2.12 的  $E(z) = \frac{z}{(z-1)(z-2)}$ 。先展開  $\frac{E(z)}{z}$ ：

$$\frac{E(z)}{z} = \frac{1}{(z-1)(z-2)} = \frac{-1}{z-1} + \frac{1}{z-2}$$

乘回  $z$ ：

$$E(z) = \frac{-z}{z-1} + \frac{z}{z-2}$$

查表得：

$$e(k) = -1 + 2^k$$

與例 2.12 用長除法得到的結果一致！

```
syms k z
Ez = z/((z-1)*(z-2));
iztrans(Ez, k)
% ans = 2^k - 1
```

**函數介紹：** `iztrans(expr, k)` —— `ztrans()` 的反運算，對符號表達式 `expr` ( 以 `z` 為自變數 ) 計算反  $z$  轉換，結果表示為以 `k` 為自變數的數列公式。這一行程式就相當於手算部分分式展開 + 查表的整個流程。

若考慮  $E_1(z) = z^{-1}E(z)$  ( 利用實數平移定理， $e_1(k) = e(k-1)u(k-1)$  )，計算  $E_1(z)/z$  的部分分式：

$$\frac{E_1(z)}{z} = \frac{1}{z(z-1)(z-2)} = \frac{1/2}{z} + \frac{-1}{z-1} + \frac{1/2}{z-2}$$

其中  $\frac{1/2}{z}$  這一項對應  $\mathcal{Z}^{-1}[\frac{1}{2}] = \frac{1}{2}\delta(k)$  ( 因為常數本身就是  $z^0 = 1$  的係數，對應脈衝在  $k = 0$  )，所以：

$$e_1(k) = (-1 + 2^{k-1})u(k-1) + \frac{1}{2}\delta(k)$$

這與直接用實數平移定理  $e_1(k) = [-1 + 2^{(k-1)}]u(k-1)$  ( 在  $k = 0$  補上修正值  $\frac{1}{2}$  ) 的結果一致。

**複數極點的情況：**若  $E(z)$  有一對共軛複數極點  $p_1 = a^T e^{\pm jbT}$  ( 即極點以  $p_1 = a^T e^{jbT}$  與其共軛  $p_1^*$  成對出現 )，部分分式會產生複數係數，但兩項相加後仍為實數。為了避免處理複數運算，可以直接用下面的公式把結果表示成**實數的正弦/餘弦**形式：

先用歐拉公式把目標數列寫成極座標形式：

$$y(k) = Aa^{kT} \cos(bkT + \varphi) = \frac{A}{2} [(a^T e^{jbT})^k e^{j\varphi} + (a^T e^{-jbT})^k e^{-j\varphi}]$$

取  $z$  轉換 ( 查表  $a^k \rightarrow \frac{z}{z-a}$  )：

$$Y(z) = \frac{z \cdot k_1}{z - p_1} + \frac{z \cdot k_1^*}{z - p_1^*}, \quad k_1 = \frac{A}{2} e^{j\varphi}$$

於是，只要從部分分式展開 ( 例如用 `residue()` ) 求出留數  $k_1$  與極點  $p_1$ ，就能反推：

$$p_1 = a^T e^{jbT} \Rightarrow aT = \ln |p_1|, \quad bT = \arg(p_1)$$

$$k_1 = \frac{A}{2} e^{j\varphi} \Rightarrow A = 2|k_1|, \quad \varphi = \arg(k_1)$$

**例 2.14：**求  $Y(z) = \frac{-3.894z}{z^2 + 0.6065}$  的反  $z$  轉換。

分母的根： $z^2 = -0.6065 \Rightarrow z = \pm j0.7788$ ，即極點  $p_1 = j0.7788$  ( 純虛數，代表沒有實部衰減、只有振盪 )。把  $Y(z)/z$  做部分分式：

$$\frac{Y(z)}{z} = \frac{-3.894}{(z - j0.7788)(z + j0.7788)}$$

留數：

$$k_1 = (z - j0.7788) \cdot \frac{-3.894}{(z - j0.7788)(z + j0.7788)} \Big|_{z=j0.7788} = \frac{-3.894}{j1.5576} = j2.5001$$

即  $k_1 = 2.5 \angle 90^\circ$ 。

```
num = [0, 0, -3.894]; den = [1, 0, 0.6065];
[r, p, k] = residue(num, den)
% r = 0+2.5001i, 0-2.5001i
% p = 0+0.7788i, 0-0.7788i
```

由  $p_1 = j0.7788 = 0.7788 \angle 90^\circ$ ：

$$aT = \ln |p_1| = \ln(0.7788) = -0.25, \quad bT = \arg(p_1) = \frac{\pi}{2}$$

由  $k_1 = 2.5 \angle 90^\circ$ ：

$$A = 2|k_1| = 5, \quad \varphi = \arg(k_1) = \frac{\pi}{2}$$

代回公式：

$$y(k) = Aa^{kT} \cos(bkT + \varphi) = 5e^{-0.25k} \cos\left(\frac{\pi}{2}k + \frac{\pi}{2}\right) = -5e^{-0.25k} \sin\left(\frac{\pi}{2}k\right)$$

(最後一步用了三角恆等式  $\cos(x + \pi/2) = -\sin x$ 。)

**⚠ 勘誤：**原書掃描文字把  $k_1$  的大小顯示為「2.590」，經 MATLAB `residue()` 與手算驗證應為 **2.5001** (約 2.5)，本筆記已訂正；同時原文  $e^{-0.25k}$  在掃描時遺失了指數符號  $e$ ，容易誤讀為  $5^{-0.25k}$ ，這裡也一併訂正標示清楚。

### 方法三：反演公式法 (留數定理)

最一般化的反  $z$  轉換技巧，直接對定義式  $e(k) = \frac{1}{2\pi j} \oint_{\Gamma} E(z)z^{k-1}dz$  用留數定理求值：

$$e(k) = \sum_{\text{所有極點}} [E(z)z^{k-1} \text{ 在各極點的留數}]$$

若  $E(z)z^{k-1}$  在  $z = a$  處有單重極點：

$$\text{留數} \Big|_{z=a} = (z - a)E(z)z^{k-1} \Big|_{z=a}$$

若  $z = a$  處是  $m$  重極點：

$$\text{留數}|_{z=a} = \frac{1}{(m-1)!} \frac{d^{m-1}}{dz^{m-1}} \left[ (z-a)^m E(z) z^{k-1} \right] \Big|_{z=a}$$

例 2.15：續例 2.12/2.13 的  $E(z) = \frac{z}{(z-1)(z-2)}$ ，兩個單重極點  $z = 1, 2$ ：

$$e(k) = \frac{z}{z-2} z^{k-1} \Big|_{z=1} + \frac{z}{z-1} z^{k-1} \Big|_{z=2} = -1 + 2^k$$

與前述方法完全一致。

例 2.16 (重根範例)：  $E(z) = \frac{z}{(z-1)^2}$  在  $z = 1$  處為 2 重極點：

$$e(k) = \frac{1}{(2-1)!} \frac{d}{dz} \left[ (z-1)^2 \cdot \frac{z}{(z-1)^2} \cdot z^{k-1} \right] \Big|_{z=1} = \frac{d}{dz} [z^k] \Big|_{z=1} = k z^{k-1} \Big|_{z=1} = k$$

```
syms k z
Ez = z/(z-1)^2;
iztrans(Ez, k)
% ans = k
```

## 方法四：離散摺積法

若  $E(z)$  可以拆成兩個較簡單函數的乘積：

$$E(z) = E_1(z) E_2(z)$$

把兩者都展開為冪級數並相乘，比較係數，可得**離散摺積和** (discrete convolution sum)：

$$e(k) = \sum_{n=0}^k e_1(n) e_2(k-n) = \sum_{n=0}^k e_1(k-n) e_2(n)$$

記為  $e(k) = e_1(k) * e_2(k)$  (這正是前面表 2-2 中「摺積  $\leftrightarrow$  相乘」性質的逆用)。

例 2.17：續  $E(z) = \frac{z}{(z-1)(z-2)} = E_1(z) E_2(z)$ ，其中：

$$E_1(z) = \frac{z}{z-1} = 1 + z^{-1} + z^{-2} + \cdots \quad (\text{即 } e_1(k) = 1, \forall k)$$

$$E_2(z) = \frac{1}{z-2} = z^{-1} + 2z^{-2} + 4z^{-3} + \cdots \quad (\text{即 } e_2(k) = 2^{k-1}, k \geq 1)$$

以  $e(3)$  為例：

$$e(3) = \sum_{n=0}^3 e_1(n)e_2(3-n) = (1)(2^2) + (1)(2) + (1)(1) + (1)(0) = 7$$

```
e1 = [1 1 1 1 1 1];  
e2 = [0 1 2 4 8 16];  
e = conv(e1, e2)  
% e = 0 1 3 7 15 31
```

**數學 ↔ MATLAB 對照：** `conv()` 這個函數同時扮演「多項式相乘」與「離散摺積」兩個角色——這不是巧合，因為多項式相乘（把同次方項的係數相加）在數學結構上與摺積求和公式完全相同，這正是  $z$  轉換把「摺積」變成「乘法」這個核心性質的具體體現。

## 七、模擬圖與訊號流程圖 (2.8)

### 1. 基本元件：時間延遲（移位暫存器）

**模擬圖**（simulation diagram，也稱方塊圖 block diagram）是差分方程式的圖形化表示。最基本的元件是**移位暫存器**（shift register）：每經過  $T$  秒，暫存器輸入一個新數值，同時把原本存的數值輸出。若輸入為  $e(k)$ ，輸出即為  $e(k-1)$ 。符號上常畫成一個標示  $T$  的方塊：

```
e(k) → [ T ] → e(k-1)
```

這個延遲元件在離散系統模擬圖中的角色，等同於**積分器**在類比系統模擬圖中的角色——類比系統的基本狀態變數存在積分器輸出，離散系統的基本狀態變數則存在延遲元件輸出。

**\*\*例：**差分方程式  $m(k) = e(k) - e(k-1) - m(k-1)$ （即例 2.10/2.11 的系統）\*\*的模擬圖，需要兩個延遲元件（一個存  $e(k-1)$ ，一個存  $m(k-1)$ ），加上加法/減法運算元件。

## 2. 訊號流程圖與轉移函數

訊號流程圖 ( signal flow graph ) 是方塊圖的另一種等價表示法，由分支 ( branch ) 與節點 ( node ) 組成：分支的輸出等於分支增益乘上輸入；節點的值等於所有流入該節點的分支訊號之和。因此流程圖與方塊圖包含完全相同的資訊，都可以用 **Mason 增益公式** 求出整體轉移函數。

延遲元件的轉移函數是  $z^{-1}$  ( 輸入  $E(z)$ ，輸出  $z^{-1}E(z)$ ，這正是實數平移定理的直接結果 )，對應到類比系統中積分器的轉移函數  $s^{-1}$ 。

## 3. 前置知識：Mason 增益公式 ( Mason' s gain formula )

本節之後 ( 以及第 5、7 章 ) 都會直接拿這個公式讀流程圖，所以先把它交代清楚。它解決的問題是：看著訊號流程圖，一次寫出輸入節點到輸出節點的整體轉移函數，不必一步一步做方塊圖化簡。

先定義四個名詞——都只跟「圖怎麼走」有關，與訊號的物理意義無關：

| 名詞                         | 定義                                                                                    |
|----------------------------|---------------------------------------------------------------------------------------|
| 前向路徑 ( forward path )      | 順著箭頭方向、從輸入節點走到輸出節點，且同一節點不重複經過的一串分支。第 $k$ 條的 <b>路徑增益</b> $P_k$ = 該路徑上所有分支增益的 <b>乘積</b> |
| 迴路 ( loop )                | 順著箭頭方向走、起點與終點是同一個節點的封閉路徑，中間節點不重複。 <b>迴路增益</b> $L_i$ = 該迴路上所有分支增益的乘積                   |
| 互不接觸 ( non-touching )      | 兩個迴路 ( 或一個迴路與一條前向路徑 ) <b>沒有共用任何節點</b>                                                 |
| 圖行列式 ( graph determinant ) | $\Delta$ ，只由迴路決定，與走哪條前向路徑無關；定義見下                                                      |

公式本體：

$$\frac{Y(z)}{U(z)} = \frac{1}{\Delta} \sum_k P_k \Delta_k$$

其中  $\Delta$  與  $\Delta_k$  定義為：

$$\Delta = 1 - \sum_i L_i + \sum_{i,j} L_i L_j - \sum_{i,j,m} L_i L_j L_m + \cdots$$

| 符號                         | 意義                                                                              |
|----------------------------|---------------------------------------------------------------------------------|
| $\sum_i L_i$               | 所有迴路增益的和                                                                        |
| $\sum_{i,j} L_i L_j$       | 所有互不接觸的迴路配對，其增益乘積之和                                                             |
| $\sum_{i,j,m} L_i L_j L_m$ | 所有兩兩互不接觸的三個迴路，其增益乘積之和；之後每項再多一個迴路，正負號逐項交替                                        |
| $\Delta_k$                 | 把所有碰到第 $k$ 條前向路徑的迴路刪掉後，用剩下的迴路依同一個式子重算一次的 $\Delta$ 。若每個迴路都碰到該路徑，則 $\Delta_k = 1$ |

⚠ 符號約定： $\Delta$  的第一項是減號 ( $1 - \sum L_i$ )，而負回授的那個  $-1$  要吃進迴路增益裡。所以單迴路負回授系統的迴路增益是  $L_1 = -GH$ ，代進去得  $\Delta = 1 - (-GH) = 1 + GH$ ——這就是為什麼熟悉的閉迴路公式長成  $1 + GH$  的樣子。第 7 章把特徵方程式寫成  $1 - G_{op}(z) = 0$  的那個「減號」也是同一個道理（那裡的  $G_{op}$  已經把負號吃進去了）。

退化情形（可以拿來檢查自己有沒有記錯）：單一前向路徑  $P_1 = G$ 、單一迴路  $L_1 = -GH$ ，且兩者相碰（故  $\Delta_1 = 1$ ）：

$$\frac{C}{R} = \frac{P_1 \Delta_1}{\Delta} = \frac{G}{1 + GH}$$

正是最基本的閉迴路轉移函數。Mason 公式就是它的一般化——前向路徑不只一條、迴路不只一個（尤其是出現互不接觸的迴路）時才真正派上用場。

#### 4. 套用：例 2.10/2.11 系統的流程圖

回到  $m(k) = e(k) - e(k-1) - m(k-1)$  的流程圖（延遲元件的分支增益為  $z^{-1}$ ）。

前向路徑有兩條（從  $E$  到  $M$ ）：

$P_1 = 1$ （ $e(k)$  直接進到  $m(k)$ ）， $P_2 = -z^{-1}$ （經過存  $e(k-1)$  的延遲，增益  $-1$ ）

迴路只有一個（ $m(k)$  經延遲元件與  $-1$  增益繞回自己）：

$$L_1 = -z^{-1} \implies \Delta = 1 - L_1 = 1 + z^{-1}$$

這個迴路與兩條前向路徑都共用  $M$  節點（都相碰），所以  $\Delta_1 = \Delta_2 = 1$ 。代入公式：

$$\frac{M(z)}{E(z)} = \frac{P_1\Delta_1 + P_2\Delta_2}{\Delta} = \frac{1 - z^{-1}}{1 + z^{-1}} = \frac{z - 1}{z + 1}$$

與例 2.11 用  $z$  轉換法直接求解得到的結果一致，驗證了「模擬圖  $\leftrightarrow$  差分方程式  $\leftrightarrow z$  轉換」三者的等價性。

## 5. 一般 $n$ 階系統的模擬圖：非最小實現 vs. 最小實現

考慮一般  $n$  階差分方程式：

$$m(k) + a_{n-1}m(k-1) + \cdots + a_0m(k-n) = b_ne(k) + b_{n-1}e(k-1) + \cdots + b_0e(k-n)$$

取  $z$  轉換後的轉移函數：

$$\frac{M(z)}{E(z)} = \frac{b_n + b_{n-1}z^{-1} + \cdots + b_0z^{-n}}{1 + a_{n-1}z^{-1} + \cdots + a_0z^{-n}}$$

直接按照差分方程式的形式畫模擬圖，會用到  $2n$  個延遲元件（ $n$  個存  $e(k-1), \dots, e(k-n)$ ，另  $n$  個存  $m(k-1), \dots, m(k-n)$ ）——這是**非最小實現**（non-minimal realization），因為理論上一個  $n$  階系統只需要  $n$  個延遲元件（狀態變數）就能完整描述。把分子分母的延遲元件「共用」，可以得到只用  $n$  個延遲元件的**最小實現**（minimal realization）。

這個「如何用最少的延遲元件（狀態）表示系統」的問題，正是引出下一節**狀態變數**方法的動機。

## 八、狀態變數 (2.9)

### 1. 為什麼需要狀態變數？

轉移函數  $G(z)$  描述的是系統的**輸入輸出關係**： $M(z) = G(z)E(z)$ 。但轉移函數看不到系統「內部」發生了什麼——例如上一節提到，同一個轉移函數可以對應到用不同數目延遲元件的模擬圖。**狀態變數方法**（state-variable method）則進一步描述系統的內部結構：除了輸入  $u_i(k)$  與輸出  $y_i(k)$ ，還引入內部的**狀態變數**  $x_i(k)$ ，代表決定系統未來行為所需的最少資訊量——只要知道目前的狀態與未來的輸入，就能唯一決定系統未來所有的狀態與輸出。

寫成向量形式，輸入向量、輸出向量、狀態向量分別為：

$$u(k) = \begin{bmatrix} u_1(k) \\ u_2(k) \\ \vdots \\ u_r(k) \end{bmatrix}, \quad y(k) = \begin{bmatrix} y_1(k) \\ y_2(k) \\ \vdots \\ y_p(k) \end{bmatrix}, \quad x(k) = \begin{bmatrix} x_1(k) \\ x_2(k) \\ \vdots \\ x_n(k) \end{bmatrix}$$

## 2. 狀態方程式的一般形式

最一般的（非線性）狀態方程式：

$$x(k+1) = f[x(k), u(k)] \quad (\text{狀態方程式})$$

$$y(k) = g[x(k), u(k)] \quad (\text{輸出方程式})$$

若系統為**線性**，上式化簡為（矩陣可能隨時間變化）：

$$x(k+1) = A(k)x(k) + B(k)u(k)$$

$$y(k) = C(k)x(k) + D(k)u(k)$$

若系統再進一步是**非時變**（time-invariant），矩陣為常數，這是本章後續討論的主要情形：

$$x(k+1) = Ax(k) + Bu(k)$$

$$y(k) = Cx(k) + Du(k)$$

符號定義：

| 符號     | 意義                          | 維度           |
|--------|-----------------------------|--------------|
| $x(k)$ | 狀態向量                        | $n \times 1$ |
| $u(k)$ | 輸入向量                        | $r \times 1$ |
| $y(k)$ | 輸出向量                        | $p \times 1$ |
| $A$    | 系統矩陣，描述狀態如何自我演化             | $n \times n$ |
| $B$    | 輸入矩陣，描述輸入如何影響狀態             | $n \times r$ |
| $C$    | 輸出矩陣，描述狀態如何組合成輸出            | $p \times n$ |
| $D$    | 直接傳遞矩陣，描述輸入直接影響輸出的部分（不經過狀態） | $p \times r$ |

### 3. 由差分方程式直接建立狀態模型

例 2.18：已知差分方程式  $y(k+2) = u(k) + 1.7y(k+1) - 0.72y(k)$ ，求狀態模型。

做法：選擇狀態變數為輸出本身及其「未來的自己」：

$$x_1(k) = y(k), \quad x_2(k) = x_1(k+1) = y(k+1)$$

則：

$$x_2(k+1) = y(k+2) = u(k) + 1.7x_2(k) - 0.72x_1(k)$$

整理成標準形式：

$$\begin{aligned} x_1(k+1) &= x_2(k) \\ x_2(k+1) &= -0.72x_1(k) + 1.7x_2(k) + u(k) \\ y(k) &= x_1(k) \end{aligned}$$

矩陣形式：

$$x(k+1) = \begin{bmatrix} 0 & 1 \\ -0.72 & 1.7 \end{bmatrix} x(k) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(k), \quad y(k) = [1 \quad 0] x(k)$$

### 4. 由轉移函數建立狀態模型：控制標準型 (CCF)

給定分子階數低於分母階數的轉移函數：

$$G(z) = \frac{Y(z)}{U(z)} = \frac{b_{n-1}z^{n-1} + b_{n-2}z^{n-2} + \cdots + b_1z + b_0}{z^n + a_{n-1}z^{n-1} + \cdots + a_1z + a_0}$$

引入輔助變數  $E(z)$ ，把  $G(z)$  拆成分子、分母各自乘上  $E(z)$ ：

$$Y(z) = (b_{n-1}z^{n-1} + \cdots + b_0)E(z), \quad U(z) = (z^n + a_{n-1}z^{n-1} + \cdots + a_0)E(z)$$

利用實數平移定理的對應關係  $z^i E(z) \leftrightarrow e(k+i)$ ，定義一串狀態變數：

$$x_1(k) = e(k), \quad x_2(k) = e(k+1), \quad \dots, \quad x_n(k) = e(k+n-1)$$

代入分母方程式整理，可得狀態方程式（前  $n-1$  個式子純粹是狀態變數之間的位移關係，最後一個式子才是真正的動態方程）：

$$\begin{aligned}
x_1(k+1) &= x_2(k) \\
x_2(k+1) &= x_3(k) \\
&\vdots \\
x_n(k+1) &= -a_0x_1(k) - a_1x_2(k) - \cdots - a_{n-1}x_n(k) + u(k)
\end{aligned}$$

寫成矩陣形式：

$$\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \\ \vdots \\ x_n(k+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & & \ddots & \vdots \\ -a_0 & -a_1 & -a_2 & \cdots & -a_{n-1} \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \\ \vdots \\ x_n(k) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix} u(k)$$

輸出方程式 ( 由分子方程式得到 )：

$$y(k) = [b_0 \quad b_1 \quad \cdots \quad b_{n-1}] \begin{bmatrix} x_1(k) \\ x_2(k) \\ \vdots \\ x_n(k) \end{bmatrix}$$

這個結構稱為**控制標準型** ( Control Canonical Form, CCF，也稱相變數標準型 phase-variable canonical form )，系統矩陣  $A$  的特殊形式 ( 下方一列是負的分母係數，其餘是移位用的 1 ) 讓我們可以「看一眼轉移函數就直接寫出狀態方程式」，在第 9、11 章的設計方法中非常有用。

**觀察  $A$  矩陣結構的規律：**最後一列 ( 或最後一行，視慣例 ) 放

$-a_0, -a_1, \dots, -a_{n-1}$  ( 分母係數，取負號 )，其餘位置是簡單的移位矩陣 ( 次對角線全 1 )； $C$  向量直接就是分子係數  $[b_0, b_1, \dots, b_{n-1}]$ 。這個「一眼看出」的規律，就是控制標準型最大的價值。

## 5. 觀測標準型 ( OCF )

另一個常用的標準型是**觀測標準型** ( Observer Canonical Form, OCF )，其模擬圖結構是 CCF 的「轉置」，同樣可以由轉移函數的係數直接排出。

**補充說明：**教材原文將 OCF 的狀態方程式推導留給讀者練習 ( 習題 2.9-2 )。以

三階系統  $G(z) = \frac{b_2z^2 + b_1z + b_0}{z^3 + a_2z^2 + a_1z + a_0}$  為例，OCF 的一組標準寫法為：

$$x(k+1) = \begin{bmatrix} -a_2 & 1 & 0 \\ -a_1 & 0 & 1 \\ -a_0 & 0 & 0 \end{bmatrix} x(k) + \begin{bmatrix} b_2 \\ b_1 \\ b_0 \end{bmatrix} u(k), \quad y(k) = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} x(k)$$

可以驗證： $C[zI - A]^{-1}B = G(z)$ （驗證方法見第十一節）。OCF 在第 9 章「觀測器設計」中特別有用，因為它讓觀測器增益的設計變得直觀。

## 6. 例 2.19：三階系統的訊號流程圖與狀態方程式

$$G(z) = \frac{Y(z)}{U(z)} = \frac{z^2 + 2z + 1}{z^3 + 2z^2 + z + 0.5}$$

⚠ 勘誤：原書掃描文字分母常數項顯示模糊（疑似遺失分數線），本筆記依後續狀態方程式矩陣中明確出現的  $-\frac{1}{2}$  訂正為 0.5。

比對一般公式  $b_0 = 1, b_1 = 2, b_2 = 1; a_0 = 0.5, a_1 = 1, a_2 = 2$ ，直接代入控制標準型公式：

$$x(k+1) = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -0.5 & -1 & -2 \end{bmatrix} x(k) + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} u(k), \quad y(k) = \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} x(k)$$

## 7. 例 2.20：分子分母同階數時的處理

$$G(z) = \frac{Y(z)}{U(z)} = \frac{b_2 z^2 + b_1 z + b_0}{z^2 + a_1 z + a_0}$$

此時分子階數等於分母階數（多了一個  $b_2$  項），會多出一條從輸入直接到輸出的路徑（direct path），對應非零的  $D$  矩陣。狀態方程式部分不變：

$$\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -a_0 & -a_1 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(k)$$

但輸出方程式要多考慮這條直接路徑。由流程圖列出  $Y(z) = b_0 X_1(z) + b_1 X_2(z) + b_2 E(z)$ ，且  $E(z) = U(z) - a_0 X_1(z) - a_1 X_2(z)$ ，代入消去  $E(z)$ ：

$$y(k) = [(b_0 - b_2 a_0) \quad (b_1 - b_2 a_1)] x(k) + b_2 u(k)$$

也就是說  $D = b_2 \neq 0$ ——只要分子分母同階，輸出方程式就一定會出現非零的直接傳遞項  $D$ ，這是判斷「要不要處理  $D$  矩陣」的簡單準則。

## 8. 例 2.21：多變數 ( MIMO ) 系統

狀態方程式的方法可以直接推廣到多輸入多輸出系統，只要從模擬圖逐一讀出即可，不需要新的理論：

$$x(k+1) = \begin{bmatrix} 0.5 & 0 \\ 1 & -1 \end{bmatrix} x(k) + \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1(k) \\ u_2(k) \end{bmatrix}$$
$$\begin{bmatrix} y_1(k) \\ y_2(k) \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} x(k) + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1(k) \\ u_2(k) \end{bmatrix}$$

**觀念解析：**這裡  $B$  是  $2 \times 2$  矩陣，因為有兩個輸入  $u_1, u_2$ ，各自可能影響兩個狀態； $C$ 、 $D$  同理，因為有兩個輸出  $y_1, y_2$ 。單變數 ( SISO ) 系統只是多變數 ( MIMO ) 系統中  $r = p = 1$  的特例。

## 9. 從轉移函數推導狀態模型的兩種程序總結

1. 代數法 ( 即上面的推導 )：把  $G(z)$  寫成 (2-51)–(2-64) 的形式，直接套公式。
2. 圖形法：(1) 用任何方便的方法畫出系統的模擬圖；(2) 每個延遲元件的輸出指定一個狀態變數；(3) 對每個延遲元件的輸入、以及系統輸出，寫出只用延遲輸出與系統輸入表示的方程式。

**重要觀念：**轉移函數是系統「唯一」的輸入輸出描述，但狀態模型 ( 內部描述 ) 不是唯一的——同一個轉移函數可以對應無限多組不同的  $(A, B, C, D)$ ，下一節將系統性地說明如何在這些等價的狀態模型之間轉換。

---

## 九、其他狀態變數形式與相似轉換 (2.10)

### 1. 串聯 ( Cascade ) 與並聯 ( Parallel ) 實現

例 2.22：續例 2.18 的轉移函數  $G(z) = \frac{Y(z)}{U(z)} = \frac{1}{z^2 - 1.7z + 0.72}$ 。

**串聯實現：**把分母因式分解  $z^2 - 1.7z + 0.72 = (z - 0.9)(z - 0.8)$ ，寫成兩個一階環節相乘：

$$\frac{Y(z)}{U(z)} = \left[ \frac{1}{z - 0.9} \right] \left[ \frac{1}{z - 0.8} \right]$$

依此畫模擬圖 ( 兩個一階子系統前後相接 )，可得狀態方程式：

$$x(k+1) = \begin{bmatrix} 0.8 & 1 \\ 0 & 0.9 \end{bmatrix} x(k) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(k), \quad y(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} x(k)$$

**並聯實現**：把轉移函數做部分分式展開：

$$\frac{Y(z)}{U(z)} = \frac{10}{z-0.9} + \frac{-10}{z-0.8}$$

依此畫模擬圖（兩個一階子系統並排、輸出相加），可得另一組狀態方程式：

$$x(k+1) = \begin{bmatrix} 0.9 & 0 \\ 0 & 0.8 \end{bmatrix} x(k) + \begin{bmatrix} 1 \\ 1 \end{bmatrix} u(k), \quad y(k) = \begin{bmatrix} 10 & -10 \end{bmatrix} x(k)$$

**一般化**：任意高階轉移函數都可以表示成低階環節的**串聯**：

$$G(z) = G_{1c}(z)G_{2c}(z) \cdots G_{nc}(z)$$

或**並聯**：

$$G(z) = G_{1p}(z) + G_{2p}(z) + \cdots + G_{np}(z)$$

若  $G(z)$  含複數極點/零點，可以選擇某些子環節為二階，避免狀態矩陣出現複數元素。

## 2. 相似轉換 ( Similarity Transformation )

**動機**：同一系統可以有無限多組狀態模型，它們之間如何系統性地互相轉換？

考慮狀態方程式：

$$x(k+1) = Ax(k) + Bu(k), \quad y(k) = Cx(k) + Du(k)$$

引入一個可逆的常數矩陣  $P$  (  $n \times n$  )，定義新的狀態向量：

$$x(k) = Pw(k)$$

代入原方程式：

$$Pw(k+1) = APw(k) + Bu(k) \Rightarrow w(k+1) = P^{-1}APw(k) + P^{-1}Bu(k)$$

$$y(k) = CPw(k) + Du(k)$$

整理成標準形式：

$$w(k+1) = A_w w(k) + B_w u(k), \quad y(k) = C_w w(k) + D_w u(k)$$

其中：

$$A_w = P^{-1}AP, \quad B_w = P^{-1}B, \quad C_w = CP, \quad D_w = D$$

這個變換稱為**相似轉換** (similarity transformation)。因為  $P$  可以任意選取 (只要可逆)，所以能產生無窮多組等價的狀態模型——這正是上一節「狀態模型不唯一」的數學根源。

符號定義：

| 符號                   | 意義                          |
|----------------------|-----------------------------|
| $P$                  | 相似轉換矩陣 ( $n \times n$ ，須可逆) |
| $w(k)$               | 轉換後的新狀態向量                   |
| $A_w, B_w, C_w, D_w$ | 轉換後的新狀態方程式矩陣                |

### 3. 特徵方程式與特徵值在相似轉換下的不變性

矩陣  $A$  的**特徵方程式** (characteristic equation) 定義為：

$$|zI - A| = 0$$

其根  $z_i$  稱為  $A$  的**特徵值** (characteristic value / eigenvalue)：

$$|zI - A| = (z - z_1)(z - z_2) \cdots (z - z_n) = 0$$

第 7 章會證明：特徵值決定了系統 (2-67) 的穩定性。

**關鍵性質**：相似轉換不改變系統矩陣的特徵方程式：

$$|zI - A_w| = |zI - P^{-1}AP| = |P^{-1}(zI - A)P| = |P^{-1}| |zI - A| |P| = |zI - A|$$

(最後一步用了  $|P^{-1}||P| = 1$ 。 ) 由此可推得三個重要性質：

1. **特徵值不變**： $A$  與  $A_w$  有完全相同的特徵值集合。
2. **行列式不變**： $|A_w| = |A| = z_1 z_2 \cdots z_n$  (行列式等於所有特徵值的乘積)。
3. **跡 (trace) 不變**： $\text{tr}(A_w) = \text{tr}(A) = z_1 + z_2 + \cdots + z_n$  (跡定義為主對角線元素之和，等於所有特徵值之和)。
4. **轉移函數不變** (第十一節將證明)： $C[zI - A]^{-1}B + D = C_w[zI - A_w]^{-1}B_w + D_w$ 。

**觀念解析：**這些不變量之所以重要，是因為它們是系統的「本質特性」——無論用哪一組狀態變數描述系統，極點位置（穩定性）、系統增益（行列式相關）等物理意義都不會因為換了一組座標（狀態變數的選取）而改變，就像換了測量單位不會改變物理定律一樣。

#### 4. 例 2.23：任選 $P$ 做相似轉換

延續例 2.22 的串聯型狀態模型：

$$x(k+1) = \begin{bmatrix} 0.8 & 1 \\ 0 & 0.9 \end{bmatrix} x(k) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(k), \quad y(k) = [1 \quad 0] x(k)$$

任選轉換矩陣：

$$P = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

用餘因子公式求逆矩陣： $P^{-1} = \frac{[\text{Cof}(P)]^T}{|P|}$ ，其中  $|P| = 1 \cdot 1 - (-1) \cdot 1 = 2$ ，

$\text{Cof}(P) = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$ （本例恰好和  $P$  本身相同，屬巧合），因此：

$$P^{-1} = \begin{bmatrix} 0.5 & 0.5 \\ -0.5 & 0.5 \end{bmatrix}$$

代入相似轉換公式：

$$A_w = P^{-1}AP = \begin{bmatrix} 1.35 & 0.55 \\ -0.45 & 0.35 \end{bmatrix}, \quad B_w = P^{-1}B = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}, \quad C_w = CP = [1 \quad -1]$$

驗證特徵值不變：

$$|zI - A| = (z - 0.8)(z - 0.9) = z^2 - 1.7z + 0.72$$

$$|zI - A_w| = z^2 - 1.7z + 0.72 \quad \checkmark \text{（相同！）}$$

驗證行列式與跡： $|A| = |A_w| = 0.72 = z_1 z_2 = (0.8)(0.9)$ ； $\text{tr}(A) = \text{tr}(A_w) = 1.7 = z_1 + z_2 = 0.8 + 0.9$ ，均成立。

#### 5. 例 2.24：用 MATLAB 做任意相似轉換

$$G(z) = \frac{5z^2 + 10.5z + 5.8}{z^3 + 1.1z^2 + 0.5z + 0.12}$$

由控制標準型公式直接寫出：

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -0.12 & -0.5 & -1.1 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad C = [5.8 \quad 10.5 \quad 5]$$

任選  $P = \begin{bmatrix} 3 & 8 & 4 \\ 0 & 7 & 5 \\ 9 & 6 & 2 \end{bmatrix}$ ，用 MATLAB 計算新的狀態模型：

```
A = [0 1 0; 0 0 1; -.12 -.5 -1.1];
B = [0 0 1]';
C = [5.8, 10.5, 5];
D = 0;
P = [3 8 4; 0 7 5; 9 6 2];
invP = inv(P)
Aw = inv(P)*A*P
Bw = inv(P)*B
Cw = C*P
Dw = D
```

函數介紹：**inv(M)** ——計算方陣 **M** 的反矩陣  $M^{-1}$ 。這是相似轉換公式  $A_w = P^{-1}AP$  中  $P^{-1}$  的直接數值計算，比手算餘因子公式（例 2.23 用的方法）快速很多，適合三階以上的矩陣。

數學 ↔ MATLAB 對照：

- $P^{-1} \leftrightarrow \text{inv}(P)$
- $A_w = P^{-1}AP \leftrightarrow \text{Aw} = \text{inv}(P)*A*P$ （矩陣乘法用 **\***，注意矩陣乘法沒有交換律，運算順序不能顛倒）

## 6. 對角化：透過特徵向量得到對角形式的狀態模型

若系統矩陣  $A$  的所有特徵值都**相異**（distinct），可以找到一組相似轉換讓  $A_w$  變成**對角矩陣**——這是最簡潔的狀態模型，因為對角矩陣代表各狀態變數彼此獨立演化，互不耦合。

考慮向量  $m_i$  與純量  $z_i$  滿足：

$$Am_i = z_i m_i$$

這就是線性代數中的**特徵值問題**： $z_i$  是  $A$  的特徵值， $m_i = [m_{1i} \ m_{2i} \ \cdots \ m_{ni}]^T$  是對應的**特徵向量**（characteristic vector / eigenvector）。把所有特徵向量當作直行，

組成**模態矩陣** ( modal matrix )  $M = [m_1 \ m_2 \ \cdots \ m_n]$ ，可以證明：

$$AM = M\Lambda \implies \Lambda = M^{-1}AM$$

其中  $\Lambda$  是以特徵值  $z_1, \dots, z_n$  為對角元素的對角矩陣。也就是說，只要取相似轉換矩陣  $P = M$  ( 模態矩陣 )，就能得到對角化的狀態模型。前提是特徵值相異，這樣才能保證特徵向量彼此線性獨立 (  $M$  才可逆 )。

**符號定義：**

| 符號                            | 意義                              |
|-------------------------------|---------------------------------|
| $m_i$                         | 對應特徵值 $z_i$ 的特徵向量               |
| $M$                           | 模態矩陣：以各特徵向量為直行組成的矩陣             |
| $\Lambda$ ( 或 $\Phi$ ，視教材記號 ) | 對角矩陣，對角線為各特徵值 $z_1, \dots, z_n$ |

## 7. 例 2.25：手算對角化

續例 2.23 的系統  $A = \begin{bmatrix} 0.8 & 1 \\ 0 & 0.9 \end{bmatrix}$ ，已知特徵值  $z_1 = 0.8$ ， $z_2 = 0.9$ 。

求  $z_1 = 0.8$  對應的特徵向量： $Am_1 = 0.8m_1$ ：

$$\begin{bmatrix} 0.8 & 1 \\ 0 & 0.9 \end{bmatrix} \begin{bmatrix} m_{11} \\ m_{21} \end{bmatrix} = 0.8 \begin{bmatrix} m_{11} \\ m_{21} \end{bmatrix} \Rightarrow 0.8m_{11} + m_{21} = 0.8m_{11}, \quad 0.9m_{21} = 0.8m_{21}$$

第二式要求  $m_{21} = 0$ ，第一式自動滿足， $m_{11}$  可任意取，令  $m_{11} = 1$ 。

求  $z_2 = 0.9$  對應的特徵向量：同理可得  $m_{22}$  任意 ( 取  $m_{22} = 1$  )，代回第一式解出  $m_{12} = 10$ 。

$$M = \begin{bmatrix} 1 & 10 \\ 0 & 1 \end{bmatrix}, \quad M^{-1} = \begin{bmatrix} 1 & -10 \\ 0 & 1 \end{bmatrix}$$

代入相似轉換：

$$\Lambda = M^{-1}AM = \begin{bmatrix} 0.8 & 0 \\ 0 & 0.9 \end{bmatrix}, \quad B_w = M^{-1}B = \begin{bmatrix} -10 \\ 1 \end{bmatrix}, \quad C_w = CM = \begin{bmatrix} 1 & 10 \end{bmatrix}$$

新的狀態模型果然是對角形式：

$$w(k+1) = \begin{bmatrix} 0.8 & 0 \\ 0 & 0.9 \end{bmatrix} w(k) + \begin{bmatrix} -10 \\ 1 \end{bmatrix} u(k), \quad y(k) = \begin{bmatrix} 1 & 10 \end{bmatrix} w(k)$$

用 MATLAB 直接算特徵值與特徵向量：

```
A = [0.8 1; 0 0.9];  
[M, LAMBDA] = eig(A)
```

函數介紹： $[V, D] = \text{eig}(A)$  ——計算方陣  $A$  的特徵向量與特徵值。回傳兩個矩陣： $V$  的每一行 (column) 是一個特徵向量 (對應特徵值  $D$  對角線上同一位置的元素)， $D$  是以特徵值為對角元素的對角矩陣，滿足  $A \cdot V = V \cdot D$ 。

驗證方式：計算  $A \cdot V - V \cdot D$ ，若結果是 (數值上) 接近零的矩陣，就代表  $V$ 、 $D$  確實滿足特徵值方程式  $AM = M\Lambda$ 。這是驗證  $\text{eig}()$  計算正確、也是驗證我們手算特徵向量正確的通用方法。

注意：MATLAB 的  $\text{eig}()$  回傳的特徵向量通常會做單位化 (長度歸一)，因此數值上會和手算「任取比例常數 = 1」的結果差一個縮放因子——但兩者代表的是同一個特徵方向，只要都拿去做相似轉換，得到的對角矩陣  $\Lambda$  (特徵值本身) 會完全相同，只有  $B_w, C_w$  的數值因縮放因子不同而有差異，這是正常現象、不代表算錯。

## 8. 例 2.26：三階系統用 MATLAB 對角化

$$G(z) = \frac{10z - 5}{z^3 - 2.1z^2 + 1.4z - 0.288}$$

⚠ 勘誤：原書掃描文字此處轉移函數的分子分母嚴重亂碼，本筆記依後續明確給

出的狀態矩陣  $A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0.288 & -1.4 & 2.1 \end{bmatrix}$  (即  $a_0 = -0.288, a_1 =$

$1.4, a_2 = -2.1$ )、 $C = [-5 \ 10 \ 0]$  (即  $b_0 = -5, b_1 = 10, b_2 = 0$ ) 反推出正確的轉移函數如上，並據此重新推導。

```

A = [0 1 0; 0 0 1; 0.288 -1.4 2.1];
B = [0 0 1]';
C = [-5 10 0];
D = 0;
[M, LAMBDA] = eig(A);
P = M;
Aw = inv(P)*A*P
Bw = inv(P)*B
Cw = C*P
Dw = D

```

執行結果會得到  $A_w = \Lambda$  為對角矩陣，對角線元素即為  $A$  的三個特徵值  $z_1 = 0.4$ ,  $z_2 = 0.8$ ,  $z_3 = 0.9$  ( 可用 `roots([1 -2.1 1.4 -0.288])` 或直接對特徵方程式  $z^3 - 2.1z^2 + 1.4z - 0.288 = (z - 0.4)(z - 0.8)(z - 0.9)$  驗證 ) 。

## 十、由狀態方程式求轉移函數 (2.11)

### 1. 方法一：從模擬圖用 Mason 增益公式

已知狀態方程式，可以先畫出對應的模擬圖，再套用 Mason 增益公式求出轉移函數 ( 公式與名詞定義見 §7.3 ) 。

例 2.27：續例 2.23 的狀態模型：

$$x(k+1) = \begin{bmatrix} 1.35 & 0.55 \\ -0.45 & 0.35 \end{bmatrix} x(k) + \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} u(k), \quad y(k) = \begin{bmatrix} 1 & -1 \end{bmatrix} x(k)$$

畫出模擬圖後套用 Mason 增益公式，計算得：

$$\frac{Y(z)}{U(z)} = \frac{z^{-2}}{1 - 1.7z^{-1} + 0.72z^{-2}} = \frac{1}{z^2 - 1.7z + 0.72}$$

正好與原始 ( 相似轉換前的 ) 系統轉移函數相同，驗證了相似轉換不改變轉移函數。

### 2. 方法二：代數法 —— $G(z) = C[zI - A]^{-1}B + D$

對狀態方程式  $x(k+1) = Ax(k) + Bu(k)$  取  $z$  轉換 ( 忽略初始條件，因為求轉移函數只關心零初始條件下的輸入輸出關係 )：

$$zX(z) - zx(0) = AX(z) + BU(z) \xrightarrow{x(0)=0} [zI - A]X(z) = BU(z)$$

解出：

$$X(z) = [zI - A]^{-1}BU(z)$$

代入輸出方程式  $Y(z) = CX(z) + DU(z)$ ：

$$Y(z) = [C[zI - A]^{-1}B + D]U(z)$$

因此轉移函數為：

$$\boxed{G(z) = C[zI - A]^{-1}B + D}$$

這是狀態空間模型轉回轉移函數最直接、最系統化的公式，對高階系統尤其實用。

**例 2.28**：續例 2.27 的模型，手算  $[zI - A]^{-1}$ ：

$$zI - A = \begin{bmatrix} z - 1.35 & -0.55 \\ 0.45 & z - 0.35 \end{bmatrix}, \quad |zI - A| = z^2 - 1.7z + 0.72$$

伴隨矩陣（餘因子矩陣的轉置）：

$$\text{Cof}[zI - A]^T = \begin{bmatrix} z - 0.35 & 0.55 \\ -0.45 & z - 1.35 \end{bmatrix}$$

（本例矩陣恰好對稱，故轉置後與原餘因子矩陣相同）

$$[zI - A]^{-1} = \frac{1}{z^2 - 1.7z + 0.72} \begin{bmatrix} z - 0.35 & 0.55 \\ -0.45 & z - 1.35 \end{bmatrix}$$

由於  $D = 0$ ：

$$\begin{aligned} G(z) &= C[zI - A]^{-1}B = \frac{1}{z^2 - 1.7z + 0.72} \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} z - 0.35 & 0.55 \\ -0.45 & z - 1.35 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \\ &= \frac{1}{z^2 - 1.7z + 0.72} \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} 0.5z + 0.1 \\ 0.5z - 0.9 \end{bmatrix} = \frac{1}{z^2 - 1.7z + 0.72} \end{aligned}$$

**MATLAB 作法（符號運算）：**

```
syms z
A = [1.35 0.55; -0.45 0.35];
B = [0.5; 0.5];
C = [1 -1];
D = 0;
Gz = C*inv(z*eye(2)-A)*B + D;
simplify(Gz)
```

函數介紹：**eye(n)** ——建立  $n \times n$  的單位矩陣  $I$ ，這裡  $z*\text{eye}(2)$  就是符號運算中的  $zI$  ( $2 \times 2$  單位矩陣乘上符號  $z$ )。

函數介紹：**simplify(expr)** ——化簡符號表達式，讓分數約分、同類項合併，使結果更接近人類習慣的簡潔形式。

執行結果為  $50/((5*z - 4)*(10*z - 9))$ ，因式分解形式和  $\frac{1}{z^2 - 1.7z + 0.72}$  是同一個有理函數 ( $5z - 4 = 5(z - 0.8)$ ， $10z - 9 = 10(z - 0.9)$ ，相乘展開後係數比例一致)。

函數介紹：**collect()**、**coeffs()**、**feval(symengine, 'numer'/'denom', expr)** ——教材提供一個自訂函數 **nice()**，內部用 **collect(old, z)** 依  $z$  的次方整理表達式，再用 **feval(symengine, 'numer', new)** 與 **'denom'** 分別取出分子、分母，最後用 **coeffs()** 取出各次方項的係數並除以最高次項係數做正規化，把 **simplify()** 給出的因式分解形式轉換回「標準多項式除以多項式」的形式，方便直接丟進 **tf()** 建立轉移函數物件：

```
[num, den] = nice(Gz, z); T = 1;
Gz = tf(num, den, T) % Gz = 1 / (z^2 - 1.7z + 0.72)
```

**MATLAB 作法 (狀態空間直接轉換)**：更常用、更簡潔的方式是使用 Control System Toolbox 內建函數：

```
A = [1.35 0.55; -0.45 0.35];
B = [0.5; 0.5];
C = [1 -1];
D = 0;
T = 1;
[num, denom] = ss2tf(A, B, C, D);
Gz = tf(num, denom, T) % Gz = 1 / (z^2 - 1.7z + 0.72)
```

函數介紹：`ss2tf(A,B,C,D)`——直接把狀態空間矩陣  $(A, B, C, D)$  轉換成轉移函數的分子、分母係數向量，內部原理正是公式  $G(z) = C[zI - A]^{-1}B + D$  的數值化實作，是本節理論最直接對應的 MATLAB 函數。

數學  $\leftrightarrow$  MATLAB 對照：

| 數學                    | MATLAB                                                                                  |
|-----------------------|-----------------------------------------------------------------------------------------|
| $zI - A$              | <code>z*eye(n)-A</code> (符號) 或內部隱含於 <code>ss2tf</code> (數值)                             |
| $[zI - A]^{-1}$       | <code>inv(z*eye(n)-A)</code>                                                            |
| $C[zI - A]^{-1}B + D$ | <code>C*inv(z*eye(n)-A)*B+D</code> (符號) 或 <code>[num,den]=ss2tf(A,B,C,D)</code> (數值，更快) |

### 3. 轉移函數在相似轉換下的不變性

因為轉移函數是系統唯一的輸入輸出描述，而相似轉換只是換一組內部座標，所以：

$$C[zI - A]^{-1}B + D = C_w[zI - A_w]^{-1}B_w + D_w$$

(證明留給讀者，可用  $A_w = P^{-1}AP$  等關係代入化簡驗證。) 這與第九節提到的「特徵值、行列式、跡」等不變量放在一起，構成相似轉換的完整不變性質清單。

## 十一、狀態方程式的解 (2.12)

### 1. 遞迴解法

假設系統非時變，且  $x(0)$  與所有  $u(j)$ ,  $j = 0, 1, 2, \dots$  已知：

$$x(k+1) = Ax(k) + Bu(k)$$

一步步代入：

$$x(1) = Ax(0) + Bu(0)$$

$$x(2) = Ax(1) + Bu(1) = A^2x(0) + ABu(0) + Bu(1)$$

$$x(3) = A^3x(0) + A^2Bu(0) + ABu(1) + Bu(2)$$

歸納出一般解：

$$x(k) = A^k x(0) + \sum_{j=0}^{k-1} A^{(k-1-j)} B u(j)$$

定義**狀態轉移矩陣** ( state transition matrix · 又稱基本矩陣 fundamental matrix ) :

$$\Phi(k) \triangleq A^k$$

則一般解可寫成 :

$$x(k) = \Phi(k)x(0) + \sum_{j=0}^{k-1} \Phi(k-1-j)Bu(j)$$

代入輸出方程式 :

$$y(k) = C\Phi(k)x(0) + \sum_{j=0}^{k-1} C\Phi(k-1-j)Bu(j) + Du(k)$$

**觀念解析**：這個解的結構分成兩部分——第一項  $\Phi(k)x(0)$  是「純粹由初始狀態演化而來」的**零輸入響應** ( zero-input response ) ；第二項的求和是「純粹由輸入激勵而來」的**零狀態響應** ( zero-state response · 一個離散摺積和 ) 。這與連續時間系統中「齊次解 + 特解」或「零輸入響應 + 零狀態響應」的概念完全對應， $\Phi(k)$  扮演的角色就相當於連續時間系統中的**狀態轉移矩陣**  $e^{At}$  。

**例 2.29** :  $G(z) = \frac{z+3}{(z+1)(z+2)}$  , 其狀態模型 ( 用第八節方法 ) 為 :

$$x(k+1) = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix} x(k) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(k), \quad y(k) = \begin{bmatrix} 3 & 1 \end{bmatrix} x(k)$$

系統靜止 (  $x(0) = 0$  ) , 輸入為單位步階  $u(k) = 1$  (  $k \geq 0$  ) 。逐步代入 :

$$x(1) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad y(1) = 3(0) + 1(1) = 1$$

$$x(2) = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad y(2) = 3(1) + 1(-2) = 1$$

同理可得  $x(3) = \begin{bmatrix} -2 \\ 5 \end{bmatrix}$  ,  $y(3) = -1$  ;  $x(4) = \begin{bmatrix} 5 \\ -10 \end{bmatrix}$  ,  $y(4) = 5$  。

```

A = [0 1; -2 -3];
B = [0; 1];
C = [3 1];
x = [0; 0];
u = 1;
for k = 0:5
    x1 = A*x + B*u;
    k
    y = C*x
    x = x1;    % 把這次算出的新狀態，變成下一輪迭代的「目前狀態」
end

```

輸出：k=0..5 對應 y = 0 1 1 -1 5 -9（因為程式先印出「用目前 x 算的 y」再更新 x，所以  $y(0) = C \cdot x(0) = 0$  對應真正的初始輸出）。

**數學 ↔ MATLAB 對照：**迴圈內  $x1 = A*x + B*u$  正是遞迴公式  $x(k+1) = Ax(k) + Bu(k)$  的直接數值實作；用「數值遞迴代入」求解狀態方程式，完全不需要事先求出  $\Phi(k)$  的封閉公式，這是計算機解狀態方程式最自然的方式。

## 2. z 轉換法求 $\Phi(k)$ 的封閉公式

遞迴法能算出任意有限步的數值，但看不出  $\Phi(k) = A^k$  關於  $k$  的封閉公式。z 轉換提供了一個系統化的求法：

令  $u(k) = 0$ ，對  $x(k+1) = Ax(k)$  取 z 轉換：

$$zX(z) - zx(0) = AX(z) \Rightarrow X(z) = z[zI - A]^{-1}x(0)$$

取反 z 轉換：

$$x(k) = \mathcal{Z}^{-1}[z[zI - A]^{-1}]x(0)$$

比對前面遞迴解中  $x(k) = \Phi(k)x(0)$ （在  $u = 0$  情況下），得到：

$$\boxed{\Phi(k) = \mathcal{Z}^{-1}[z[zI - A]^{-1}]}$$

也就是說：先求  $[zI - A]^{-1}$ 、乘上  $z$ 、再對矩陣的每個元素分別做反 z 轉換，就能得到  $\Phi(k) = A^k$  的封閉公式，不需要真的去計算  $A$  的  $k$  次方！

**例 2.30：**續例 2.29 的  $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$ ：

$$zI - A = \begin{bmatrix} z & -1 \\ 2 & z+3 \end{bmatrix}, \quad |zI - A| = z^2 + 3z + 2 = (z+1)(z+2)$$

$$z[zI - A]^{-1} = \frac{z}{(z+1)(z+2)} \begin{bmatrix} z+3 & 1 \\ -2 & z \end{bmatrix}$$

對每個元素做部分分式展開再反  $z$  轉換 ( 例如  $\frac{z(z+3)}{(z+1)(z+2)} = \frac{2z}{z+1} - \frac{z}{z+2} \rightarrow 2(-1)^k - (-2)^k$  ) , 得 :

$$\Phi(k) = \begin{bmatrix} 2(-1)^k - (-2)^k & (-1)^k - (-2)^k \\ -2(-1)^k + 2(-2)^k & -(-1)^k + 2(-2)^k \end{bmatrix}$$

驗證：把  $\Phi(1)$  代入應該等於  $A$  本身；把上面例 2.29 遞迴算出的  $x(1), x(2), \dots$  拿來和  $\Phi(k)x(0) + \sum \dots$  比對，結果會一致。

### 3. 用電腦做數值方法

$$x(k+1) = Ax(k) + Bu(k)$$

這類方程式很適合直接用電腦遞迴求解 ( 如例 2.29 的 MATLAB 迴圈 ) , 不需要真的求出  $\Phi(k)$  的封閉公式。另一個電腦方法是直接數值計算  $\Phi(k) = A^k$  :

```
Phi = A^k; % 這裡 ^ 是「矩陣次方」，不是逐元素次方 .^
```

⚠ 特別注意 ( 第二十四節提到的常見錯誤 ) :  $A^k$  代表矩陣  $A$  自乘  $k$  次 (  $A \times A \times \dots \times A$  ) , 這是矩陣次方運算；若誤寫成  $A.^k$  ( 逐元素次方 ) , MATLAB 會把矩陣中每一個元素各自獨立做  $k$  次方，結果完全不同、也沒有  $\Phi(k) = A^k$  這個物理意義。這個方法的缺點是只能得到「某個特定  $k$ 」的數值結果，看不出  $\Phi(k)$  作為  $k$  的函數的封閉公式 ( 高階系統即使用  $z$  轉換法也很難得到封閉公式，這時數值方法反而更實際 ) 。

### 4. 狀態轉移矩陣的性質

由  $x(k) = \Phi(k)x(0)$  及  $\Phi(k) = A^k$  , 可以推導出三個重要性質：

性質一：令  $k = 0$  :

$$\Phi(0) = A^0 = I \quad (\text{單位矩陣})$$

性質二：指數律：

$$\Phi(k_1 + k_2) = A^{k_1+k_2} = A^{k_1} A^{k_2} = \Phi(k_1)\Phi(k_2)$$

性質三：

$$\Phi(-k) = A^{-k} = [A^k]^{-1} = \Phi^{-1}(k) \iff \Phi(k) = \Phi^{-1}(-k)$$

例 2.31： $A = \begin{bmatrix} 1 & 0 \\ 0 & 0.5 \end{bmatrix}$ （對角矩陣，方便驗證）：

$$\Phi(k) = A^k = \begin{bmatrix} 1^k & 0 \\ 0 & 0.5^k \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0.5^k \end{bmatrix}$$

驗證性質一： $\Phi(0) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I \checkmark$

驗證性質二： $\Phi(k_1 + k_2) = \begin{bmatrix} 1 & 0 \\ 0 & 0.5^{k_1+k_2} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0.5^{k_1} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 0.5^{k_2} \end{bmatrix} = \Phi(k_1)\Phi(k_2) \checkmark$

驗證性質三： $\Phi^{-1}(-k) = \begin{bmatrix} 1 & 0 \\ 0 & 0.5^{-k} \end{bmatrix}^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 0.5^k \end{bmatrix} = \Phi(k) \checkmark$

## 十二、線性時變系統 (2.13)

前面各節都假設系統為**非時變**（矩陣  $A, B, C, D$  為常數）。若系統為**線性時變**（矩陣隨時間  $k$  改變），狀態方程式為：

$$x(k+1) = A(k)x(k) + B(k)u(k), \quad y(k) = C(k)x(k) + D(k)u(k)$$

**遞迴推導**：設初始時刻為  $k_0$ ， $x(k_0)$  已知，逐步代入：

$$x(k_0 + 1) = A(k_0)x(k_0) + B(k_0)u(k_0)$$

$$x(k_0 + 2) = A(k_0 + 1)A(k_0)x(k_0) + A(k_0 + 1)B(k_0)u(k_0) + B(k_0 + 1)u(k_0 + 1)$$

歸納後定義時變的**狀態轉移矩陣**：

$$\Phi(k, k_0) \triangleq \begin{cases} A(k-1)A(k-2)\cdots A(k_0) = \prod_{j=k_0}^{k-1} A(j), & k > k_0 \\ I, & k = k_0 \end{cases}$$

一般解為：

$$x(k) = \Phi(k, k_0)x(k_0) + \sum_{j=k_0}^{k-1} \Phi(k, j+1)B(j)u(j)$$

$$y(k) = C(k)\Phi(k, k_0)x(k_0) + \sum_{j=k_0}^{k-1} C(k)\Phi(k, j+1)B(j)u(j) + D(k)u(k)$$

時變狀態轉移矩陣的性質（對照非時變版本的性質一~三）：

$$\Phi(k_0, k_0) = I, \quad \Phi(k_2, k_1)\Phi(k_1, k_0) = \Phi(k_2, k_0), \quad \Phi(k_1, k_2) = \Phi^{-1}(k_2, k_1)$$

**觀念解析：**因為時變系統的矩陣會隨時間改變，狀態轉移矩陣必須在每個時間點重新計算，不能像非時變系統一樣直接套用  $A^k$  這個簡單的封閉公式。當系統退化為非時變（ $A$  不再是  $k$  的函數）時， $\Phi(k, k_0) = A^{k-k_0}$ ，恰好化簡回 (2-92) 式的  $\Phi(k) = A^k$ ——這說明非時變系統只是時變系統的一個特例。

## 十三、本章總結 (2.14)

本章建立了離散時間系統分析的完整數學工具箱：

1. 離散時間系統由**差分方程式**描述，是連續時間系統微分方程式的離散對應。
2. **z 轉換**把差分方程式變成代數方程式，是求解離散系統的核心工具，並具備線性、平移、初值/終值等一系列實用性質。
3. **反 z 轉換**有冪級數法、部分分式法、留數（反演公式）法、離散摺積法四種技巧，各有適用場合。
4. **模擬圖與訊號流程圖**把差分方程式視覺化，也是引出**狀態變數**方法的自然橋樑。
5. **狀態變數模型**提供比轉移函數更豐富的系統內部描述，且透過**相似轉換**可以在無限多組等價的狀態模型間轉換（包括最實用的控制標準型、觀測標準型、對角化形式）。
6. 狀態方程式的解由**狀態轉移矩陣**  $\Phi(k) = A^k$  決定，可用遞迴法（電腦數值解）或 z 轉換法（封閉公式）求得。

這些工具將是後續章節（取樣與重建、開迴路/閉迴路系統分析、穩定性、數位控制器設計）的數學基礎。

## 十四、本章 MATLAB / Octave 函數總整理

| 函數                                | 功能               | 對應數學運算                                                 |
|-----------------------------------|------------------|--------------------------------------------------------|
| <code>syms</code>                 | 宣告符號變數           | 使變數可用於符號 ( 代數 ) 運算                                     |
| <code>ztrans(e, k)</code>         | 符號 z 轉換          | $E(z) = \mathcal{Z}[e(k)]$                             |
| <code>iztrans(E, k)</code>        | 符號反 z 轉換         | $e(k) = \mathcal{Z}^{-1}[E(z)]$                        |
| <code>pretty(expr)</code>         | 美化顯示符號表達式        | ( 顯示用，不影響運算 )                                          |
| <code>residue(num, den)</code>    | 部分分式展開 ( 多項式係數 ) | 求留數 $r$ 、極點 $p$ 、直接項 $k$                               |
| <code>conv(a, b)</code>           | 多項式相乘 / 離散摺積     | $a(z) \cdot b(z)$ 或 $e_1(k) * e_2(k)$                  |
| <code>tf(num, den, T)</code>      | 建立離散時間轉移函數模型     | $G(z) = \frac{\text{num}(z)}{\text{den}(z)}$ ，取樣週期 $T$ |
| <code>ilaplace(E)</code>          | 符號反拉普拉斯轉換        | $e(t) = \mathcal{L}^{-1}[E(s)]$                        |
| <code>subs(expr, old, new)</code> | 符號代換             | 例如把 $t$ 代換成 $kT$                                       |
| <code>for k = a:b ... end</code>  | 迴圈               | 差分方程式的逐次代入求解                                           |
| <code>inv(M)</code>               | 矩陣求逆             | $M^{-1}$                                               |
| <code>eye(n)</code>               | 建立單位矩陣           | $I_{n \times n}$                                       |
| <code>[V, D] = eig(A)</code>      | 特徵值與特徵向量分解       | $AV = VD$ ，即 $V$ 為特徵向量、 $D = \Lambda$ 為特徵值對角矩陣         |
| <code>simplify(expr)</code>       | 化簡符號表達式          | 分數約分、合併同類項                                             |
| <code>collect(expr, z)</code>     | 依變數 $z$ 整理多項式    | 把展開式按次方排序                                              |
| <code>coeffs(expr)</code>         | 取出多項式係數          | 提取 $a_n, \dots, a_0$                                   |
| <code>ss2tf(A, B, C, D)</code>    | 狀態空間轉轉移函數        | $G(z) = C[zI - A]^{-1}B + D$                           |

| 函數                                            | 功能         | 對應數學運算                     |
|-----------------------------------------------|------------|----------------------------|
| <code>A^k</code> ( 矩陣次方，注意非 <code>.^</code> ) | 矩陣的 $k$ 次方 | $\Phi(k) = A^k$ ( 狀態轉移矩陣 ) |

---

## 十五、延伸閱讀 ( 本章對應的 `.m` 與 `.ipynb` )

---

- [chp2.m](#)：可直接執行的精簡版 MATLAB/Octave 程式碼，涵蓋本章所有數值範例。
- [chp2.ipynb](#)：教學版 Jupyter Notebook，以 Markdown Cell + Code Cell 交錯呈現，逐步解說每個 MATLAB 函數與數學公式的對應關係。